ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO

KẾT QUẢ NGHIÊN CỨU KHOA HỌC

HIỆN THỰC CƠ CHẾ DYNAMIC PARTIAL RECONFIGURATION CHO THIẾT KẾ MÃ HÓA AES-256 TRÊN FPGA

Người hướng dẫn: PGS.TS. TRƯƠNG QUANG VINH
ThS. TRƯƠNG VĂN CƯƠNG

Sinh viên thực hiện:

STT	MSSV	Họ và tên
1	2451167	Lê Minh Thịnh
2	2451101	Nguyễn Bảo Long
3	2451155	Ông Bảo Thành

Thành phố Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 5

1.1 Đặt vấn đề 5

1.2 Mục tiêu nghiên cứu 5

1.3 Phạm vi nghiên cứu 5

1.4 Cấu trúc báo cáo 5

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 7

2.1 Tổng quan AES-256 7

2.2 Các phép biến đổi AES 7

2.3 Key Expansion của AES-256 8

2.4 Các phương pháp hiện thực S-box 8

2.5 Nguyên lý Dummy Noise 9

2.6 Nền tảng SoC, AXI4-Lite và Timing 9

2.7 Cơ chế DPR và Replica 10

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG MÃ HÓA AES-256 12

3.1 Kiến trúc tổng quan hệ thống 12

3.1.1 Yêu cầu thiết kế . . . 12

3.1.2 Kiến trúc PS–PL . . . 12

3.1.3 Nguyên lý hoạt động tổng thể . . . 13

3.2 Thiết kế chi tiết lõi mã hóa AES-256 14

3.2.1 Cấu trúc tổng thể AES-256 Core . . . 14

3.2.2 Thiết kế Cipher Datapath . . . 15

3.2.3 Thiết kế Key Expansion . . . 16

3.2.4 Thiết kế logic điều khiển Round . . . 18

3.2.5 Trình tự xử lý và đồng bộ tín hiệu . . . 19

3.3 Thiết kế các biến thể phần cứng AES-256 21

3.3.1 Biến thể LUT S-box . . . 21

3.3.2 Biến thể BRAM S-box . . . 22

3.3.3 Biến thể Composite Field . . . 23

3.3.4 Biến thể Dummy Noise . . . 24
3.3.5 So sánh cấu trúc các biến thể . . . 26
3.4 Thiết kế giao tiếp PS–PL 27
3.4.1 Kiến trúc kết nối PS–PL . . . 27
3.4.2 Custom AXI4-Lite Controller . . . 28
3.4.3 Register Map và tín hiệu điều khiển . . . 28
3.4.4 Handshake và trình tự trao đổi dữ liệu . . . 29
3.5 Thiết kế kiến trúc DPR và Replica 31
3.5.1 Kiến trúc Top-level DPR . . . 31
3.5.2 Kiến trúc Top-level Replica . . . 32
3.5.3 Giao diện chung và cơ chế chuyển đổi biến thể . . . 34
CHƯƠNG 4: HIỆN THỰC TRÊN FPGA VÀ ARM/LINUX 36
4.1 Môi trường và quy trình hiện thực 36
4.2 Tích hợp hệ thống SoC trong Vivado 36
4.3 Hiện thực kiến trúc DPR 36
4.4 Hiện thực kiến trúc Replica 38
4.5 Triển khai trên Kria KV260 40
4.6 Chương trình điều khiển ARM/Linux 40
CHƯƠNG 5: KIỂM CHỨNG VÀ ĐÁNH GIÁ AES-256 42
5.1 Phương pháp kiểm chứng 42
5.2 Kiểm chứng bằng RTL Simulation 42
5.3 Kiểm chứng chức năng trên Kria KV260 43
5.4 Đánh giá tài nguyên và Timing sau Implementation 46
5.4.1 Mức sử dụng tài nguyên của toàn thiết kế . . . 46
5.4.2 Phân tách tài nguyên lõi mã hóa và phần giao tiếp . . . 46
5.4.3 Kết quả Timing sau Routing . . . 47
5.5 Nhận xét tổng hợp 47
CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ 49
6.1 Thiết lập thực nghiệm 49

6.2 Đánh giá quá trình chuyển đổi giữa các biến thể 49
6.3 Đánh giá trên cùng workload AES-256 50
6.4 Công suất ước lượng sau Implementation 52
6.5 Thảo luận và giới hạn kết quả 52
TÀI LIỆU THAM KHẢO 53

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề

Advanced Encryption Standard (AES) là thuật toán mã hóa khối được NIST chuẩn hóa theo FIPS 197 [1]. Trên FPGA, AES-256 có thể được triển khai bằng nhiều cấu trúc phần cứng: S-box tra bảng bằng LUT, ROM đồng bộ trên BRAM hoặc mạch tính toán Composite Field. Các lựa chọn này thực hiện cùng chức năng mã hóa nhưng khác nhau về tài nguyên và lịch xử lý.

Một hệ thống sử dụng nhiều biến thể cần quyết định cách bố trí chúng trên FPGA. Dynamic Partial Reconfiguration (DPR) cho phép các Reconfigurable Module (RM) lần lượt sử dụng một vùng Reconfigurable Partition (RP), trong khi kiến trúc Replica giữ tất cả các lõi trong cùng một cấu hình và lựa chọn bằng logic điều khiển. DPR tiết kiệm tài nguyên được chiếm dụng đồng thời nhưng phát sinh chi phí nạp Partial Bitstream; Replica không phải tái cấu hình khi chuyển lõi nhưng cần tài nguyên cho cả bốn biến thể. Các nghiên cứu của Hettwer và cộng sự, Bow và cộng sự đã khảo sát sự đa dạng hiện thực trong bối cảnh bảo vệ mạch mật mã [10, 11]; báo cáo này tập trung vào khía cạnh thiết kế và vận hành trên phần cứng.

Đề tài xây dựng một hệ thống AES-256 trên AMD Kria KV260, gồm bốn biến thể LUT, BRAM, Composite và Dummy Noise, cùng hai phương án tích hợp DPR và Replica. Chương trình C trên ARM/Linux trao đổi dữ liệu bằng AXI4-Lite, quản lý lựa chọn biến thể và kiểm tra kết quả mã hóa. Mục tiêu thực nghiệm là đối chiếu tính đúng đắn, mức sử dụng tài nguyên, khả năng đáp ứng Timing, thời gian chuyển đổi và hiệu năng workload trên cùng bo mạch.

1.2 Mục tiêu nghiên cứu

Nhóm đặt ra bốn mục tiêu: (i) hiện thực và kiểm chứng bốn lõi AES-256 có cùng giao diện chức năng; (ii) xây dựng Custom AXI4-Lite Controller để trao đổi Key, Plaintext, Ciphertext và trạng thái giữa PS–PL; (iii) triển khai DPR với một RP và Replica với bốn Core thường trú; (iv) thực nghiệm trên KV260 để đối chiếu tính đúng đắn và chi phí chuyển đổi theo cùng điều kiện. Đóng góp chính của báo cáo nằm ở kiến trúc RTL, cơ chế phối hợp phần mềm–phần cứng và dữ liệu thực nghiệm thu thập từ hai phương án.

1.3 Phạm vi nghiên cứu

Thiết kế thực hiện AES-256 *Encryption* với khóa 256 bit, Plaintext/Ciphertext 128 bit và 14 Round. Hệ thống sử dụng KV260, Vivado và Ubuntu 22.04; giao tiếp PS–PL là AXI4-Lite Memory-Mapped I/O, chưa dùng DMA. Kiểm chứng được thực hiện bằng RTL Simulation và các bộ vector Encryption của NIST. Thử nghiệm chuyển đổi ngẫu nhiên và AES workload được áp dụng cho cả DPR và Replica.

Báo cáo không hiện thực chức năng Decryption, các chế độ nhiều khối như CBC/CTR hoặc một giao thức trao đổi dữ liệu bảo mật hoàn chỉnh. Khối Dummy Noise được khảo sát về hiện thực và công suất ước lượng, nhưng chưa có phép đo side-channel để kết luận hiệu quả chống tấn công. Các chỉ số Power lấy từ Vivado được phân biệt với số đo công suất thực tế trên bo mạch.

1.4 Cấu trúc báo cáo

Chương 2 tóm tắt cơ sở AES-256, S-box, giao tiếp AXI và DPR. Chương 3 trình bày thiết kế RTL, lịch cập nhật dữ liệu theo Clock và kết nối DPR–Replica. Chương 4 mô tả hiện thực trong Vivado và triển khai trên KV260. Chương 5 trình bày kiểm chứng chức năng, tài nguyên và Timing. Chương 6 phân tích switching latency, workload và công suất ước lượng.

Câu hỏi nghiên cứu và cách đánh giá. Đề tài tập trung vào ba câu hỏi: (1) một giao diện AXI4-Lite có thể phục vụ cả bốn biến thể AES-256 khi được triển khai bằng hai cách tổ chức phần cứng hay không; (2) cách đặt S-box và Key Expansion làm thay đổi số chu kỳ xử lý, LUT, FF và BRAM như thế nào; (3) chi phí chuyển RM của DPR ảnh hưởng đến hiệu năng ứng dụng so với lựa chọn Core ở Replica đến mức nào. Các câu hỏi được đánh giá bằng ba loại bằng chứng tương ứng: sơ đồ RTL/điều khiển, báo cáo Utilization–Timing sau Routing, và log/CSV trên KV260.

Bảng 1: Liên hệ giữa đóng góp thiết kế và bằng chứng sử dụng trong báo cáo.

Nội dung thực hiện	**Bằng chứng và chỉ tiêu đánh giá**
Thiết kế bốn lõi AES-256	Đường dữ liệu, FSM, Key Expansion, chu kỳ xử lý và mô phỏng RTL
Hai Top-level DPR–Replica	Ranh giới RP, HOLD/RELEASE, Start Decoder và Output MUX
Tích hợp ARM/Linux–AXI	Địa chỉ thanh ghi, START/READY/DONE, log nạp Bitstream và Ciphertext
Kiểm chứng và thực nghiệm	405 NIST vector/Core, Timing–Utilization, switching latency và workload

Số liệu chính được ghi theo cấu hình và điều kiện thử nghiệm thay vì suy luận rằng một biến thể là tối ưu trên mọi FPGA. Ví dụ, một báo cáo WNS dương ở 100 MHz xác nhận ràng buộc Timing được đáp ứng nhưng không trực tiếp đo thời gian nạp Partial Bitstream; phép kiểm chứng mã hóa PASS cũng không chứng minh hiệu quả chống side-channel. Cách phân loại bằng chứng này được áp dụng thống nhất khi trình bày kết quả Chương 5 và Chương 6.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan AES-256

Theo FIPS 197, AES là mã khối có State 128 bit, biểu diễn bằng ma trận 4×4 byte theo thứ tự cột. Phiên bản AES-256 có khóa 256 bit ($N_k = 8$ word), sử dụng 14 Round ($N_r = 14$) và 15 Round Key, gồm một khóa cho Initial AddRoundKey và 14 khóa cho các Round [1]. State được ánh xạ từ 16 byte Plaintext $p_0, \ldots, p_{15}$ theo $s_{r,c} = p_{4c+r}$.

$$S^{(0)} = P \oplus K^{(0)}. \tag{1}$$

Với $1 \le r \le 13$, một Round gồm SubBytes (SB), ShiftRows (SR), MixColumns (MC) và AddRoundKey:

$$S^{(r)} = \mathrm{MC}(\mathrm{SR}(\mathrm{SB}(S^{(r-1)}))) \oplus K^{(r)}. \tag{2}$$

Round 14 bỏ MixColumns:

$$C = \mathrm{SR}(\mathrm{SB}(S^{(13)})) \oplus K^{(14)}. \tag{3}$$

Các biểu thức mô tả thứ tự phép toán, không ấn định số chu kỳ Clock. Thiết kế của nhóm sử dụng Datapath lặp, tức một bộ logic được tái sử dụng qua các Round; cách cấp Round Key và chốt State quyết định lịch xử lý thực tế.

Bảng 2: Các bước thuật toán và đường xử lý tương ứng trong AES-256.

Bước	**Phép toán**	**Ý nghĩa khi hiện thực RTL**
Khởi tạo	$P \oplus K^{(0)}$	Chốt State sau khi nhận Start
Round 1–13	SB, SR, MC, XOR Key	Datapath có thể hồi tiếp; Key Expansion phải cấp đúng Round Key
Round 14	SB, SR, XOR Key	MUX bypass MixColumns trước AddRoundKey
Hoàn tất	State cuối → Ciphertext	Chỉ công bố dữ liệu khi READY/DONE hợp lệ

Theo cách thiết kế Iterative, số Round của AES là thuộc tính thuật toán, còn số Clock do cấu trúc chốt dữ liệu quyết định. LUT có thể tính cả một Round giữa hai cạnh, BRAM phải thêm pha đọc S-box đồng bộ. Đây là cơ sở giải thích kết quả 15 và 29 chu kỳ trong Testbench, thay vì quy kết thời gian xử lý chỉ từ số Round.

2.2 Các phép biến đổi AES

SubBytes. Mỗi byte được thay bằng giá trị S-box theo FIPS 197. S-box gồm phép nghịch đảo nhân trên $\mathrm{GF}(2^8)$ và phép biến đổi affine. Trường AES sử dụng đa thức bất khả quy

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\texttt{0x11B}). \tag{4}$$

Từ góc độ RTL, 16 byte của State có thể tra song song bằng 16 S-box tổ hợp hoặc truy cập 16 ROM đồng bộ; hai lựa chọn khác nhau về số pha xử lý.

ShiftRows. Các hàng của State lần lượt dịch vòng trái $0, 1, 2, 3$ byte. Nếu $s_{r,c}$ là phần tử trước phép hoán vị thì $s'_{r,c} = s_{r,(c+r) \bmod 4}$. Phép biến đổi này có thể được biểu diễn bằng các kết nối và phép gán byte trong Datapath.

MixColumns. Mỗi cột State được biến đổi tuyến tính trong GF(2^8) theo ma trận hệ số 02, 03, 01:

$$\begin{bmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{bmatrix}. \tag{5}$$

Với byte a, nhân với 02 được hiện thực bằng xtime(a) (dịch trái và XOR có điều kiện với `0x1B`); nhân 03 bằng xtime(a) $\oplus\, a$. Vì vậy, mô tả phần cứng chủ yếu dùng XOR và logic bit.

AddRoundKey. State 128 bit được XOR với Round Key 128 bit. Phép toán này phải nhận khóa đúng Round và có đường bypass MixColumns khi $r = 14$. Phần còn lại của báo cáo tập trung vào cách RTL sắp xếp các khối trên cùng Datapath và chốt đầu ra ở cạnh Clock.

Các ràng buộc mà Datapath phải giữ. S-box nhận một byte và trả một byte mà không phụ thuộc vị trí State. ShiftRows chỉ thay đổi thứ tự byte, vì vậy có thể được thể hiện bằng nối dây và phép gán tổ hợp. MixColumns phải đọc bốn byte sau ShiftRows trong cùng một cột; AddRoundKey nhận đúng thứ tự byte của Round Key. Một sai sót trong thứ tự byte tại ranh giới giữa các phép toán có thể tạo Ciphertext sai ngay cả khi từng S-box được cài đặt đúng. Do đó, khi kiểm tra RTL, cần đối chiếu mapping các bus `[127:0]` theo thứ tự byte từ file NIST đến State và từ State cuối đến thanh ghi Ciphertext.

2.3 Key Expansion của AES-256

Key Expansion sinh 60 word 32 bit $w[0], \ldots, w[59]$ từ tám word khóa ban đầu; Round Key thứ r gồm $w[4r]$ đến $w[4r+3]$. Theo FIPS 197 [1], với $8 \le i < 60$:

$$w[i] = w[i-8] \oplus t_i, \quad t_i = \begin{cases} \text{SubWord}(\text{RotWord}(w[i-1])) \oplus \text{Rcon}[i/8], & i \bmod 8 = 0, \\ \text{SubWord}(w[i-1]), & i \bmod 8 = 4, \\ w[i-1], & \text{trường hợp khác.} \end{cases} \tag{6}$$

RotWord dịch vòng một byte trong word; SubWord thay thế S-box trên bốn byte; Rcon là hằng số Round. Cách hiện thực không nhất thiết lưu toàn bộ 60 word: RTL của nhóm giữ cửa sổ tám word gần nhất trong thanh ghi 256 bit rồi cập nhật theo từng nhóm bốn word. Hai nửa cửa sổ đóng vai trò nhóm khóa cũ và nhóm khóa gần nhất. Cách phối hợp cập nhật khóa với State Register được phân tích tại Mục 3.2.

Cách tổ chức khóa ở cấp thanh ghi. Nếu lưu cả 60 word 32 bit, tổng bộ nhớ lịch khóa là 1 920 bit. Với cửa sổ tám word (256 bit) như thiết kế của nhóm, khi sinh một nhóm bốn word mới, nửa khóa gần nhất trở thành nửa cũ và bốn word mới được đưa vào nửa thấp. Quá trình này tuân theo quan hệ phụ thuộc $w[i-8]$ và $w[i-1]$ mà không cần giữ tất cả 15 Round Key. Bốn word của nhóm mới được tạo theo Cascade XOR, trong đó kết quả word trước là một đầu vào của word sau. Điều này giảm dung lượng thanh ghi nhưng tạo đường logic tổ hợp có nhiều tầng XOR, cần được kiểm tra Timing sau Routing.

Ở AES-256, tám word gốc tương ứng hai Round Key đầu tiên. Khi phân tích lịch chốt khóa, cần tách hai khái niệm: *Round Key được sử dụng ở cạnh hiện tại* và *Round Key đang được tính để ghi cho cạnh sau.* Nếu hai thành phần cập nhật bằng nonblocking assignment tại cùng cạnh Clock, Cipher Core sử dụng giá trị thanh ghi cũ theo ngữ nghĩa RTL. Quan hệ chi tiết của cửa sổ khóa được trình bày và minh họa ở Mục 3.2.3.

2.4 Các phương pháp hiện thực S-box

Ba biến thể LUT, BRAM và Composite Field có cùng ánh xạ S-box nhưng dùng phần cứng khác nhau. LUT lưu 256 giá trị byte dưới dạng hàm tra cứu tổ hợp, có thể được ánh xạ vào logic LUT/Distributed ROM. BRAM sử

dụng ROM đồng bộ; địa chỉ đọc được nhận theo Clock và giá trị đầu ra sẵn sàng ở pha kế tiếp. Composite Field phân rã phép nghịch đảo của $\mathrm{GF}(2^8)$ thành các phép tính trên trường con, thay thao tác tra bảng bằng mạng logic XOR/AND [2].

Với biểu diễn $\mathrm{GF}(2^8) \cong \mathrm{GF}((2^4)^2)$, một phần tử được viết $u = az + b$, $a, b \in \mathrm{GF}(2^4)$ và $z^2 + z + \lambda = 0$. Phép nghịch đảo có thể được tính qua chuẩn $N(u) = b^2 + ab + \lambda a^2$ thuộc trường con và ánh xạ ngược về cơ sở AES. Việc chọn ma trận ánh xạ là thành phần của kiến trúc S-box cụ thể, không phải giá trị có thể chọn tùy ý.

Bảng 3: Đặc điểm các phương án S-box trên FPGA.

Tiêu chí	**LUT**	**BRAM**	**Composite**
Tổ chức	ROM logic tổ hợp	ROM đồng bộ	Phép toán trên trường con
Thời điểm đầu ra	Cùng pha tổ hợp	Sau pha đọc theo Clock	Theo độ sâu logic tổ hợp
Tài nguyên chính	LUT	BRAM và điều khiển	LUT, cổng XOR/AND

Trong thiết kế thực tế, số S-box được nhân bản, số pha xử lý và cách tối ưu của Vivado quyết định tài nguyên và Timing; không thể suy ra ưu thế chỉ từ tên phương pháp. Mục 3.3 trình bày cấu trúc sử dụng trong RTL của nhóm.

Nguồn khác biệt về chu kỳ và tài nguyên. Việc lưu bảng 256×8 bit là dung lượng thông tin 2 048 bit, không tương đương một phần tử LUT vật lý. Một State cần 16 phép tra byte, Key Expansion còn có bốn phép tra cho SubWord. Nếu triển khai 20 đường tra song song, thiết kế phải cấp đủ tài nguyên logic hoặc cổng đọc bộ nhớ cho chúng. BRAM của Xilinx có số cổng đọc hữu hạn nên việc đáp ứng 20 đường tra thường liên quan đến nhiều bản sao ROM hoặc tổ chức chia sẻ theo thời gian. Trong báo cáo Vivado của nhóm, biến thể BRAM sử dụng Block RAM vật lý và có số LUT thấp hơn LUT S-box; kết luận này dựa trên Utilization thực tế, không chỉ trên thuộc tính được ghi trong mã RTL.

Trong Composite Field, các khối Mapping, phép nghịch đảo ở trường con và Inverse Mapping tạo thành một đường tổ hợp. Thiết kế không cần pha đọc ROM, nhưng độ sâu XOR/AND có thể lớn hơn LUT tra bảng. Vì vậy, sự khác biệt giữa ba biến thể phải được phân tích đồng thời qua sơ đồ Datapath, số chu kỳ và báo cáo Timing, thay vì đánh giá một phương án là nhanh hơn chỉ vì không dùng bộ nhớ.

2.5 Nguyên lý Dummy Noise

Dummy Noise bổ sung một nhánh hoạt động chuyển mạch độc lập với kết quả Ciphertext. Trong CMOS, thành phần công suất động thường được mô tả gần đúng bởi

$$P_{\mathrm{dyn}} \approx \alpha C V^2 f, \tag{7}$$

trong đó α là hệ số hoạt động chuyển mạch. Biến thể Noise của nhóm bổ sung Correlated-Power-Noise Generator (CPNG) hoạt động song song với Datapath AES và không ghi dữ liệu ngược vào đường tạo Ciphertext. Các thanh ghi giả ngẫu nhiên và điều kiện Enable được mô tả trong Mục 3.3. Việc tăng Switching Activity không tự chứng minh hiệu quả chống Side-Channel Attack; báo cáo chỉ đánh giá chức năng, tài nguyên và công suất ước lượng, chưa thực hiện CPA hoặc đo vết công suất thực tế [5, 11].

2.6 Nền tảng SoC, AXI4-Lite và Timing

Kria KV260 sử dụng Zynq UltraScale+ MPSoC, gồm Processing System (PS) chạy ARM/Linux và Programmable Logic (PL) chứa lõi AES. Phần mềm trên PS thực hiện Memory-Mapped I/O tới một AXI4-Lite Slave trong PL [6, 8]. Với bus dữ liệu 32 bit, khóa AES-256 được truyền qua tám lần ghi thanh ghi, Plaintext qua bốn lần ghi và Ciphertext qua bốn lần đọc.

AXI4-Lite có năm kênh độc lập AW, W, B, AR và R. Trên mỗi kênh, giao dịch được chấp nhận ở cạnh Clock khi cả `VALID` và `READY` bằng 1; dữ liệu phải giữ ổn định cho đến thời điểm chấp nhận. Giao thức AXI không mô tả thời gian xử lý bên trong AES Core: dữ liệu được tập hợp tại Register Map rồi phát thành bus Key/Plaintext và xung Start.

Ở mức lõi, $T_{\text{clk}} = 1/f_{\text{clk}}$ và độ trễ phụ thuộc số chu kỳ xử lý. Ở mức ứng dụng, thời gian còn bao gồm ghi/đọc MMIO và kiểm tra trạng thái. Timing Summary của Vivado sử dụng WNS/TNS để kiểm tra các ràng buộc đã khai báo [7]; WNS dương không phải số đo trực tiếp độ trễ mã hóa. Các miền Clock, Reset và Register Map cụ thể được phân tích tại Mục 3.4.

Bảng 4: Các kênh AXI4-Lite liên quan đến Custom AES Controller.

Kênh	**Handshake**	**Vai trò**
Write Address (AW)	AWVALID/AWREADY	Chuyển Offset thanh ghi ghi
Write Data (W)	WVALID/WREADY	Chuyển dữ liệu 32 bit, WSTRB chọn byte
Write Response (B)	BVALID/BREADY	Xác nhận thao tác ghi đã được Slave tiếp nhận
Read Address (AR)	ARVALID/ARREADY	Chuyển Offset thanh ghi cần đọc
Read Data (R)	RVALID/RREADY	Trả dữ liệu STATUS hoặc Ciphertext

AXI4-Lite không hỗ trợ burst nhiều beat như AXI4 đầy đủ; tám word KEY được ghi trong tám giao dịch logic riêng. Mặc dù AW và W là hai kênh độc lập, bộ điều khiển phải bảo đảm giải mã chính xác địa chỉ tương ứng dữ liệu ghi. Ở mức AES, xung Start cần được tạo một lần cho mỗi lệnh COMMAND hợp lệ và không được tái kích hoạt chỉ vì Master giữ VALID trên một kênh. Đây là điểm phân biệt giữa Handshake bus và Handshake tác vụ mã hóa.

2.7 Cơ chế DPR và Replica

Trong quy trình DFX, Static Design duy trì các khối giao tiếp và điều khiển, còn một Reconfigurable Partition (RP) chứa Reconfigurable Module (RM) đã được nạp tại thời điểm chạy. Các RM thay thế phải tương thích giao diện tại ranh giới RP; Pblock xác định vùng tài nguyên vật lý dành cho RP [9]. Để nạp RM mới, phần mềm hoàn tất giao dịch hiện tại, cách ly các đường giao tiếp, nạp Partial Bitstream rồi giải phóng cách ly trước khi Start tác vụ mới.

Kiến trúc Replica giữ bốn Core cùng hiện diện và lựa chọn bằng Start Decoder/Output MUX. Hai kiến trúc có thể dùng chung giao diện Key, Plaintext, Start, Ciphertext và Ready; khác biệt nằm ở bước chuyển biến thể. Với DPR, bước chuyển thực hiện tại mặt phẳng cấu hình FPGA; với Replica, bước chuyển là giao dịch điều khiển ở mức logic.

Chuỗi lựa chọn giả ngẫu nhiên với seed cố định được dùng trong thực nghiệm để kiểm tra các hướng chuyển đổi giữa bốn biến thể. Việc đổi thứ tự biến thể không phải bằng chứng chống tấn công kênh kề; trong báo cáo này, nó được sử dụng để đánh giá tính đúng đắn và độ trễ của cơ chế chuyển đổi.

Bảng 5: Sự khác biệt cơ bản của hai phương án triển khai.

Đặc trưng	**DPR**	**Replica**
Biến thể thường trú	Một RM trong RP	Bốn Core đồng thời
Đổi biến thể	Nạp Partial Bitstream	Đổi Start Decoder/Output MUX
Giữ giao diện AXI	Static logic vẫn hoạt động	Cùng Register Map cho bốn Core
Chi phí chính	Thời gian tái cấu hình	Tài nguyên đồng thời của nhiều Core

Một tình huống đặc biệt xuất hiện khi RM đang bị thay thế: dữ liệu và trạng thái đầu ra vùng RP không được

phép xem là hợp lệ. Thiết kế cần cô lập các bus tại ranh giới vùng tái cấu hình, đồng thời đưa logic RM vào Reset trong khoảng HOLD. Cách nhóm hiện thực bằng `RM_HOLD` được trình bày trong Mục 3.5; cơ chế này khác với điều khiển `rm_select` của Replica.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG MÃ HÓA AES-256

3.1 Kiến trúc tổng quan hệ thống

Hệ thống AES-256 được thiết kế trên nền tảng AMD Kria KV260, kết hợp khả năng điều khiển của Processing System (PS) với tài nguyên phần cứng trong Programmable Logic (PL). Mục tiêu của thiết kế là thực hiện mã hóa AES-256, đồng thời hỗ trợ triển khai và chuyển đổi giữa bốn biến thể phần cứng gồm LUT, BRAM, Composite Field và Dummy Noise.

Kiến trúc hệ thống được tổ chức theo hướng tách biệt phần mềm điều khiển, giao tiếp dữ liệu và lõi mã hóa. Chương trình C chạy trên ARM/Linux đảm nhiệm việc cung cấp dữ liệu, phát lệnh và đọc kết quả. Trong khi đó, các phép biến đổi của thuật toán AES-256 được thực hiện trực tiếp bằng phần cứng FPGA.

3.1.1 Yêu cầu thiết kế

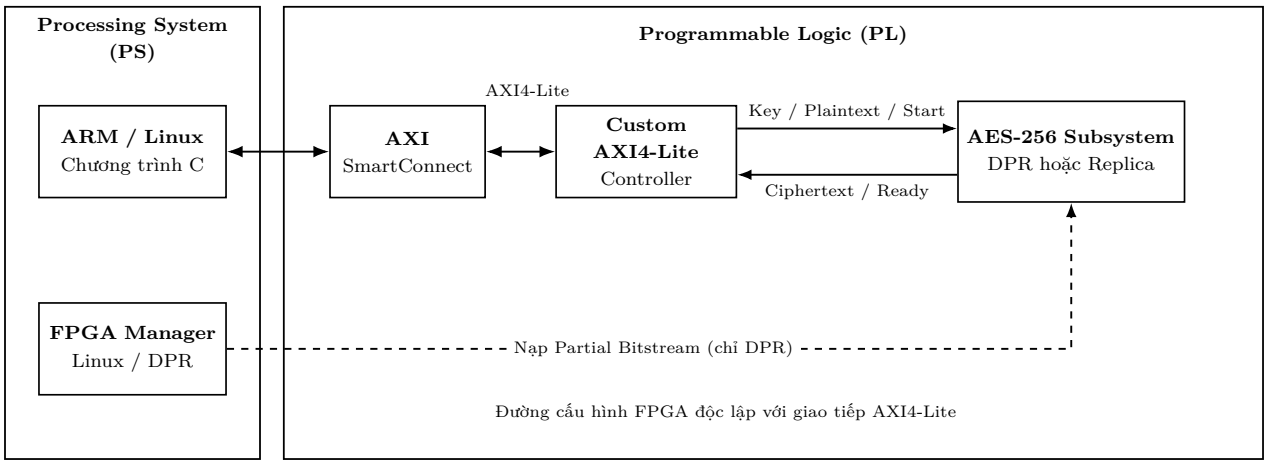
Hệ thống được xây dựng dựa trên ba yêu cầu chính: bảo đảm chức năng mã hóa AES-256, hỗ trợ giao tiếp PS–PL và duy trì khả năng thay đổi biến thể phần cứng.

Trước hết, AES Core phải tiếp nhận Encryption Key 256 bit và Plaintext 128 bit, thực hiện 14 Round theo chuẩn FIPS 197 và tạo ra Ciphertext 128 bit. Các biến thể phần cứng có thể khác nhau về cấu trúc nội bộ nhưng phải tạo ra cùng kết quả mã hóa đối với một bộ dữ liệu đầu vào.

Bên cạnh đó, hệ thống cần có giao diện cho phép phần mềm trên PS ghi dữ liệu, khởi tạo quá trình mã hóa và nhận kết quả từ PL. Đồng thời, các biến thể phải sử dụng giao diện chức năng tương thích để có thể triển khai trong kiến trúc DPR hoặc Replica mà không cần thay đổi toàn bộ hệ thống điều khiển.

3.1.2 Kiến trúc PS–PL

Kiến trúc tổng quan của hệ thống được trình bày trong Hình 1. Trong đó, PS thực thi chương trình C trên Linux, còn PL chứa các khối AXI SmartConnect, AXI4-Lite Controller và AES-256 Core.



Hình 1: Kiến trúc tổng quan hệ thống AES-256 trên nền tảng Kria KV260.

Trong đường giao tiếp dữ liệu, PS truy cập AXI4-Lite Controller thông qua AXI SmartConnect. Controller đóng vai trò trung gian, chuyển các giao dịch đọc/ghi thanh ghi thành những tín hiệu điều khiển và dữ liệu phù hợp với AES Core. Nhờ đó, các biến thể AES-256 không cần trực tiếp xử lý giao thức AXI4-Lite.

Đối với cơ chế DPR, AES Core được tổ chức thành Reconfigurable Module (RM), trong khi AXI Controller và các khối giao tiếp chung được duy trì trong Static Design. Việc nạp Partial Bitstream được quản lý từ phía

Linux thông qua cơ chế cấu hình FPGA, tách biệt với đường giao tiếp dữ liệu AXI4-Lite. Hình trên biểu diễn các đường chức năng ở mức hệ thống, không mô tả chi tiết phần cứng cấu hình bên trong SoC.

3.1.3 Nguyên lý hoạt động tổng thể

Quá trình mã hóa bắt đầu khi chương trình C trên PS ghi Encryption Key và Plaintext vào các thanh ghi AXI4-Lite Controller. Do giao diện dữ liệu sử dụng bus 32 bit, Encryption Key được chia thành tám word và Plaintext được chia thành bốn word trước khi truyền đến phần cứng.

Sau khi dữ liệu được thiết lập, phần mềm ghi lệnh Start để yêu cầu AES Core bắt đầu mã hóa. Controller phát tín hiệu điều khiển tương ứng, trong khi AES Core thực hiện các phép biến đổi và Key Expansion theo kiến trúc của biến thể đang hoạt động. Khi có kết quả hợp lệ, Controller lưu Ciphertext và cập nhật trạng thái để chương trình C có thể đọc lại thông qua AXI4-Lite.

Đối với DPR, quá trình thay đổi biến thể được thực hiện bằng cách nạp Partial Bitstream tương ứng vào vùng tái cấu hình. Trong kiến trúc Replica, các biến thể cùng tồn tại trên FPGA và được lựa chọn bằng logic điều khiển. Hai phương án này sử dụng chung nguyên tắc trao đổi dữ liệu với phần mềm nhưng khác nhau về tổ chức phần cứng và chi phí chuyển đổi.

Miền Clock, Reset và ranh giới module. Hệ thống có hai lớp giao tiếp cần được phân biệt: tầng AXI4-Lite sử dụng `S_AXI_ACLK`/`S_AXI_ARESETN`, còn cổng lõi AES sử dụng `clk_sys`/`rst_n`. Trong các module AES, State Register, Key Register, bộ đếm Round và các S-box đồng bộ của BRAM đều cập nhật theo cạnh Clock của lõi. Các khối SubBytes–ShiftRows–MixColumns–AddRoundKey ở LUT là đường tổ hợp giữa hai lần cập nhật thanh ghi, không phải bốn tác vụ lấy bốn chu kỳ. Vì vậy, cùng một Clock có thể kích hoạt nhiều thanh ghi đồng thời, trong khi giá trị tổ hợp lan truyền giữa các cạnh Clock.

Bảng 6: Phân định Clock, Reset và chức năng tại các ranh giới RTL.

Khối	**Clock/Reset thể hiện trong RTL**	**Hành vi cần kiểm tra**
AXI4-Lite Slave	`S_AXI_ACLK`, `S_AXI_ARESETN`	Chốt giao dịch MMIO, START và trạng thái; không tính Round AES
Cipher/Key Expansion LUT	`clk_sys`, `rst_n`	State/Key cập nhật cùng cạnh; phép biến đổi tổ hợp trong khoảng giữa cạnh
AES BRAM	`clk_sys`, `rst_n`	Pha đọc ROM đồng bộ và pha cập nhật Round được tách theo FSM
DPR RM	`clk_sys`, `app_rst_n`	Reset RM chịu ảnh hưởng của `rm_hold`; Static AXI vẫn duy trì
Replica	Clock/Reset chung cấp đến bốn lõi	Start được giải mã riêng; Ciphertext/Ready được MUX theo lõi chọn

Các báo cáo Timing có hai tên Clock `clk_pl_0` và `clk_pl_1` cùng chu kỳ ràng buộc 10 ns. Chỉ từ giá trị chu kỳ không thể kết luận chúng cùng một miền đồng bộ. Việc chứng minh quan hệ đồng bộ giữa `S_AXI_ACLK` và `clk_sys` phải dựa vào sơ đồ net Clock của Block Design và ràng buộc Timing cuối cùng; nếu dùng hai miền độc lập, các tín hiệu Start/Ready đa bit cần cơ chế CDC thích hợp. Báo cáo hiện tại không tuyên bố đã có cầu CDC độc lập khi chưa có netlist làm bằng chứng.

Hợp đồng giao tiếp giữa các khối. Top Wrapper giữ một bộ cổng chức năng chung cho bốn biến thể, còn cách cấp Ready và thời điểm có Ciphertext phụ thuộc từng vi kiến trúc. Mỗi giao dịch phải tuân thủ thứ tự: Key/Plaintext ổn định trước Start; Start chỉ được chấp nhận khi Core sẵn sàng; Selection của Replica hoặc RM đang hoạt động của DPR được giữ ổn định đến khi dữ liệu đầu ra hợp lệ. Bảng này là các *điều kiện giao dịch*, không phải giả định rằng mọi Core hoàn thành cùng số chu kỳ.

3.2 Thiết kế chi tiết lõi mã hóa AES-256

Lõi mã hóa AES-256 được xây dựng bằng SystemVerilog theo kiến trúc xử lý lặp (Iterative Architecture), trong đó cùng một Datapath được sử dụng qua nhiều Round thay vì triển khai toàn bộ 14 Round thành các tầng phần cứng riêng biệt. Phương án này cho phép tái sử dụng các khối biến đổi dữ liệu và phối hợp trực tiếp với quá trình sinh Round Key.

Trong thiết kế của nhóm, kiến trúc LUT được sử dụng làm cơ sở để phân tích tổ chức Cipher Datapath, Key Expansion và logic điều khiển. Các biến thể Composite Field và Dummy Noise kế thừa cách xử lý lặp, trong khi biến thể BRAM có sự điều chỉnh về trình tự xử lý để đáp ứng cơ chế đọc S-box đồng bộ. Những khác biệt này sẽ được phân tích riêng trong Mục 3.3.

3.2.1 Cấu trúc tổng thể AES-256 Core

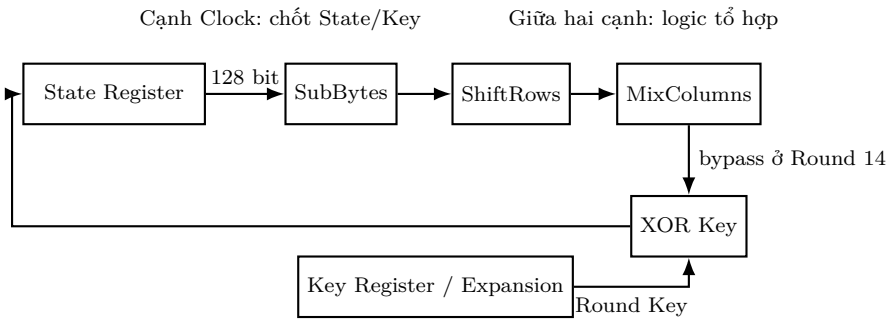
Trong mã nguồn `aes256_lut.sv`, module `aes256_lut` đóng vai trò Top Wrapper của lõi mã hóa. Module này tiếp nhận Encryption Key 256 bit, Plaintext 128 bit, Clock, Reset và tín hiệu yêu cầu mã hóa `cipher_en`. Hai tín hiệu đầu ra gồm `cipher_text` 128 bit và `cipher_ready`, dùng để cung cấp kết quả và trạng thái hoạt động cho khối điều khiển bên ngoài. Bên trong Top Wrapper, thiết kế được chia thành hai module chức năng chính:

- **AES-256 Cipher Core:** Thực hiện các phép biến đổi trên State 128 bit, quản lý State Register, theo dõi số Round và cung cấp trạng thái hoàn tất quá trình mã hóa.
- **AES-256 Key Expansion:** Tiếp nhận Encryption Key 256 bit, lưu trữ các word khóa cần thiết và sinh Round Key 128 bit theo trình tự xử lý của Cipher Core.

Hai module được đồng bộ thông qua các tín hiệu `round_num` và `rkey_en`. Cipher Core cung cấp chỉ số Round hiện tại và tín hiệu cho phép cập nhật Key Schedule, còn Key Expansion trả về `round_key_out` có độ rộng 128 bit.

Ở bước khởi tạo, Cipher Core sử dụng 128 bit đầu tiên của Encryption Key để thực hiện Initial AddRoundKey. Sau đó, các Round Key tiếp theo được cung cấp bởi Key Expansion, bảo đảm mỗi Round sử dụng đúng phần khóa tương ứng. Việc tách Cipher Core và Key Expansion thành hai module giúp phân chia rõ chức năng xử lý dữ liệu và sinh khóa. Đồng thời, Top Wrapper duy trì giao diện đầu vào và đầu ra thống nhất, thuận tiện cho việc tích hợp các biến thể AES-256 vào hệ thống DPR hoặc Replica.

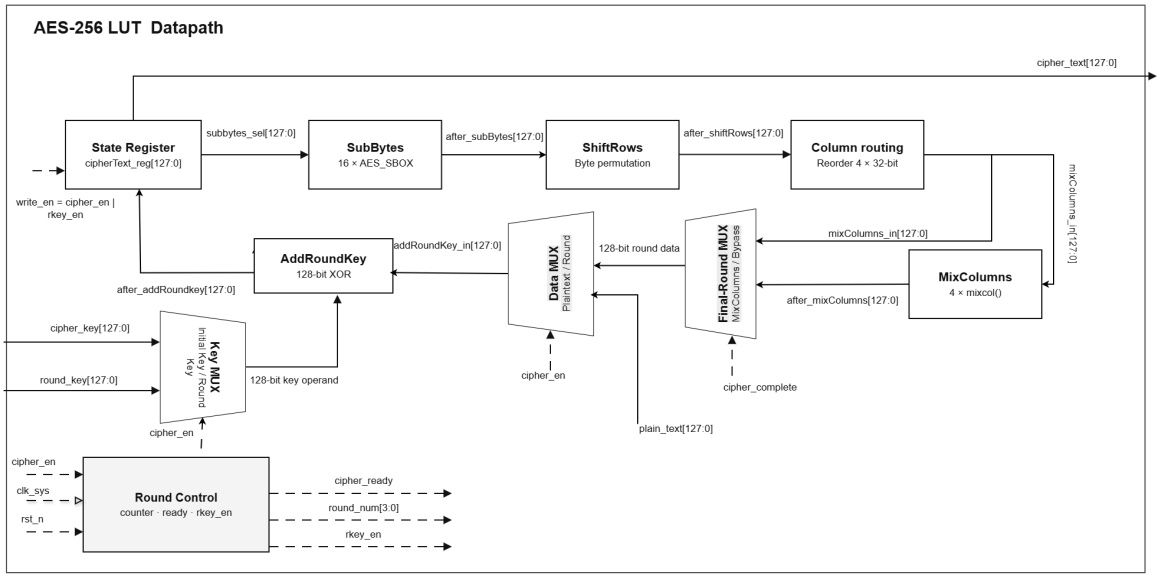
Ranh giới tổ hợp và tuần tự trong lõi. Trong kiến trúc lặp, việc sử dụng một Datapath cho 14 Round không có nghĩa các toán tử hoạt động nối tiếp qua bốn Clock. Tại một Round thông thường, State Register giữ $S^{(r-1)}$ ổn định giữa hai cạnh Clock; S-box, ShiftRows, MixColumns và XOR với Round Key tính toán tổ hợp để tạo $S^{(r)}$. Tại cạnh kế tiếp, State Register nhận dữ liệu mới và cùng Datapath bắt đầu xử lý Round sau. Key Expansion có các thanh ghi độc lập, nhưng sử dụng cùng chỉ số `round_num` và tín hiệu Enable để không cấp nhầm khóa vòng.



Hình 2: Quan hệ dữ liệu giữa hai cạnh Clock trong một Round LUT; đường Init/MUX được lược bớt để làm rõ hồi tiếp.

3.2.2 Thiết kế Cipher Datapath

Cipher Datapath của biến thể LUT được tổ chức theo kiến trúc Rolling, sử dụng một State Register 128 bit để lưu trữ dữ liệu trung gian. Sau mỗi Round, kết quả được ghi trở lại thanh ghi này và tiếp tục đi qua các khối biến đổi ở Round kế tiếp. Cấu trúc Datapath được trình bày trong Hình 3.



Hình 3: Kiến trúc Cipher Datapath của AES-256 LUT

Datapath gồm các khối SubBytes, ShiftRows, MixColumns, AddRoundKey và logic lựa chọn dữ liệu. Khác với kiến trúc Fully Unrolled, thiết kế không nhân bản toàn bộ chuỗi biến đổi cho 14 Round. Thay vào đó, State được cập nhật sau mỗi cạnh Clock và quay lại Datapath để thực hiện Round tiếp theo.

State Register và đường dữ liệu hồi tiếp

Trong RTL, tín hiệu `cipherText_reg` được sử dụng làm State Register 128 bit. Đây là thanh ghi lưu kết quả của Initial AddRoundKey và các Round tiếp theo, đồng thời được nối trực tiếp đến đầu ra `cipher_text`. Tại thời điểm nhận lệnh bắt đầu, dữ liệu ghi vào State Register được xác định bằng phép XOR giữa Plaintext và Round Key khởi tạo:

$$S_0 = P \oplus K^{(0)},$$

trong đó P là Plaintext 128 bit và $K^{(0)}$ là 128 bit đầu tiên của Encryption Key.

Sau bước khởi tạo, dữ liệu được lấy từ State Register, đưa qua các phép biến đổi và ghi kết quả trở lại cùng thanh ghi. Cơ chế này tạo thành đường hồi tiếp cho toàn bộ quá trình mã hóa.

Trong RTL, việc lựa chọn giữa Plaintext ban đầu và dữ liệu của các Round được điều khiển bằng tín hiệu `cipher_en`. Khi `cipher_en` được kích hoạt, Datapath sử dụng Plaintext và Initial Round Key. Trong các chu kỳ xử lý tiếp theo, đầu vào phép AddRoundKey được lấy từ kết quả của các phép biến đổi trên State hiện tại.

Khối SubBytes

Khối SubBytes của biến thể LUT sử dụng module `aes_sbox_lut_rom`, trong đó bảng S-box được mô tả dưới dạng ROM gồm 256 phần tử, mỗi phần tử có độ rộng 8 bit. Thiết kế tạo 16 instance S-box thông qua cấu trúc `generate`, tương ứng với 16 byte của State 128 bit. Nhờ đó, toàn bộ các phép thay thế byte có thể được thực hiện song song trong Datapath tổ hợp, không cần chia State thành những nhóm byte để tra cứu tuần tự.

Trong mã nguồn, tín hiệu `subbytes_sel` thực hiện sắp xếp lại các byte đầu vào cho các instance S-box. Sau khi tra bảng, các byte đầu ra được tập hợp trong tín hiệu `after_subBytes`. Thuộc tính `rom_style = "distributed"` được sử dụng để định hướng Vivado triển khai các bảng tra bằng tài nguyên Distributed ROM. Tuy nhiên, số LUT vật lý cuối cùng vẫn cần được xác định từ báo cáo Synthesis.

Khối ShiftRows

ShiftRows được hiện thực bằng các phép gán tổ hợp để thay đổi vị trí của các byte. Khối này không cần thực hiện phép toán số học và không sử dụng một State Register riêng. Do cách bố trí byte trong RTL, các phép gán tại `subbytes_sel`, `after_shiftRows` và `mixColumns_in` phối hợp với nhau để đưa dữ liệu về đúng thứ tự xử lý theo cột của AES. Vì vậy, khi phân tích sơ đồ ở mức RTL cần xét toàn bộ đường định tuyến byte thay vì chỉ một nhóm phép gán riêng lẻ. Cách tổ chức bằng kết nối tổ hợp giúp thực hiện thao tác hoán vị mà không cần bổ sung một chu kỳ xử lý dành riêng cho ShiftRows.

Khối MixColumns

Trong thiết kế, State sau ShiftRows được chia thành bốn nhóm 32 bit tương ứng với bốn cột của ma trận State. Mỗi cột được xử lý bằng hàm `mixcol`, sử dụng các hàm `mul2` và `mul3` để thực hiện phép nhân trong $GF(2^8)$. Bốn phép tính MixColumns được mô tả độc lập và có thể được tổng hợp thành các mạng logic hoạt động song song. Kết quả được ghép thành bus `after_mixColumns` 128 bit để đưa đến phép AddRoundKey. Đối với Round cuối cùng, AES-256 không thực hiện MixColumns [1]. Vì vậy, Datapath có logic lựa chọn dựa trên tín hiệu `cipher_complete`. Khi bộ đếm đạt giá trị 14, dữ liệu được lấy từ `mixColumns_in`, tức State đã được sắp xếp đúng thứ tự sau ShiftRows, và bỏ qua kết quả MixColumns.

Khối AddRoundKey

AddRoundKey được thực hiện bằng phép XOR giữa dữ liệu State và Round Key 128 bit. Trong RTL, tín hiệu `addRoundKey_in` lựa chọn dữ liệu đầu vào, sau đó kết quả được xác định bởi:

$$\texttt{after_addRoundkey} = \texttt{addRoundKey_in} \oplus K,$$

với K là Initial Round Key khi bắt đầu mã hóa hoặc Round Key do Key Expansion cung cấp trong các Round tiếp theo. Sau khi tính toán, kết quả được ghi vào `cipherText_reg` tại cạnh lên của Clock nếu tín hiệu `cipher_en` hoặc `rkey_en` được kích hoạt. Như vậy, đối với biến thể LUT, các phép biến đổi của một Round được tổ chức thành đường logic tổ hợp nằm giữa hai lần cập nhật State Register. Thiết kế ưu tiên tái sử dụng Datapath theo thời gian, thay vì nhân bản toàn bộ phần cứng cho từng Round. Có thể biểu diễn hoạt động cập nhật State như sau:

$$S_r = \begin{cases} \text{MixColumns}\left(\text{ShiftRows}\left(\text{SubBytes}(S_{r-1})\right)\right) \oplus K^{(r)}, & 1 \le r \le 13, \\ \text{ShiftRows}\left(\text{SubBytes}(S_{13})\right) \oplus K^{(14)}, & r = 14. \end{cases}$$

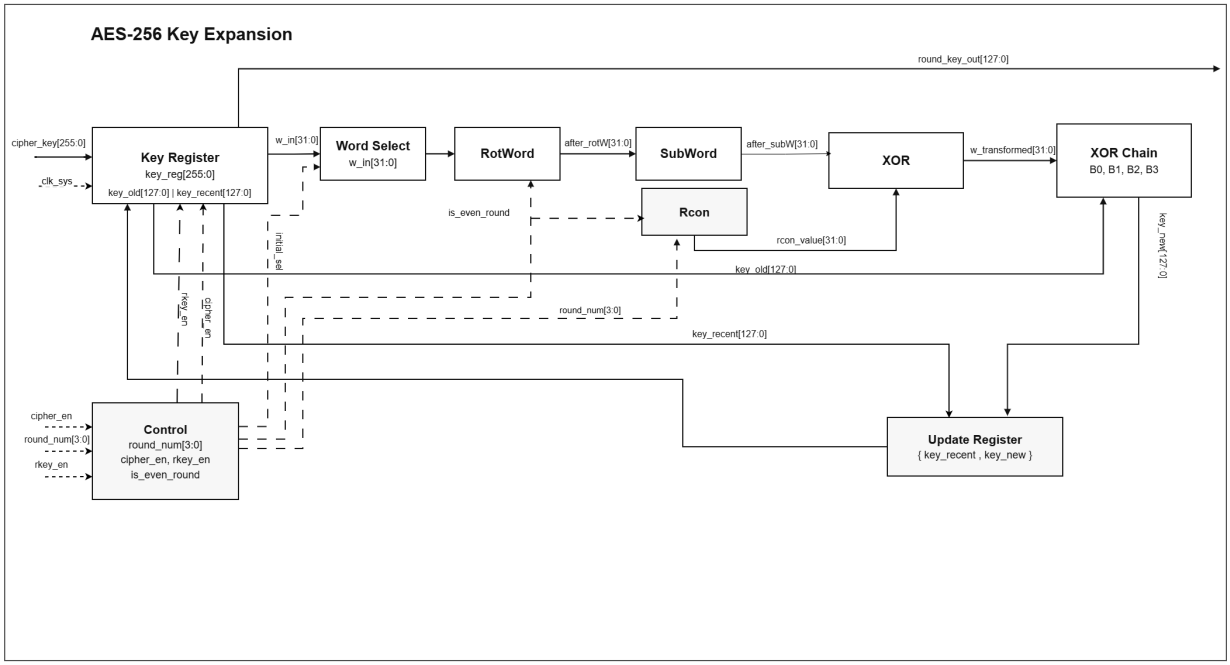
Kết quả cuối cùng S_{14} chính là Ciphertext 128 bit của phép mã hóa.

3.2.3 Thiết kế Key Expansion

Key Expansion có nhiệm vụ cung cấp Round Key 128 bit tương ứng với từng giai đoạn xử lý của Cipher Core. Đối với AES-256, quá trình sinh khóa phụ cần sử dụng tám word 32 bit gần nhất, thay vì chỉ bốn word như AES-128. Trong biến thể LUT, module `aes256_key_expansion` sử dụng thanh ghi `key_reg` 256 bit để lưu trữ một cửa sổ gồm tám word khóa. Thanh ghi được chia thành hai nhóm:

$$K_{\text{old}} = \texttt{key_reg[255:128]},$$
$$K_{\text{recent}} = \texttt{key_reg[127:0]}.$$

Trong đó, K_{old} chứa bốn word cũ hơn, còn K_{recent} chứa bốn word gần nhất. Tại thời điểm khởi tạo, hai nhóm này lần lượt tương ứng với $K^{(0)}$ và $K^{(1)}$ của Encryption Key ban đầu. Hình 4 trình bày cấu trúc sinh khóa được xây dựng từ RTL.



Hình 4: Cấu trúc Key Expansion sử dụng cửa sổ khóa 256 bit

Tổ chức cửa sổ khóa 256 bit

Khi `cipher_en` được kích hoạt, toàn bộ Encryption Key được nạp vào `key_reg`. Sau đó, tại mỗi lần cập nhật khóa, nhóm 128 bit gần nhất được dịch sang nửa trên của thanh ghi, còn nhóm 128 bit mới sinh được đưa vào nửa dưới:

$$K_{\text{window}}^{\text{next}} = \{K_{\text{recent}}, K_{\text{new}}\}.$$

Trong RTL, quá trình cập nhật được điều khiển bởi tín hiệu `rkey_en`. Cách tổ chức này giúp Key Expansion chỉ cần lưu tám word gần nhất để tạo khóa cho những Round tiếp theo, thay vì lưu toàn bộ 60 word của Key Schedule trong các thanh ghi riêng biệt.

RotWord, SubWord và Rcon

Tín hiệu `w_in` được lấy từ 32 bit cuối cùng của K_{recent}. Word này được đưa qua khối RotWord và SubWord để tạo giá trị trung gian phục vụ sinh bốn word mới. Đối với AES-256, cách biến đổi phụ thuộc vào vị trí của các word cần sinh. Trong RTL, tín hiệu `is_even_round` được xác định từ bit thấp nhất của `round_num`. Với quy ước Round của thiết kế, các Round lẻ sử dụng RotWord, SubWord và Rcon để chuẩn bị nhóm word mới, còn các Round chẵn chỉ sử dụng SubWord. Giá trị trung gian được mô tả:

$$T_r = \begin{cases} \text{SubWord}\left(\text{RotWord}(W_r)\right) \oplus \text{Rcon}\left[\dfrac{r+1}{2}\right], & r \text{ lẻ}, \\ \text{SubWord}(W_r), & r \text{ chẵn}, \end{cases}$$

trong đó W_r là word 32 bit cuối cùng của K_{recent} tại Round r.

Khối SubWord của biến thể LUT sử dụng bốn instance `aes_sbox_lut_rom`, cho phép thay thế đồng thời bốn

byte của word đầu vào. Kết quả được lưu trên bus `after_subW` 32 bit. Rcon được cung cấp thông qua hàm `aes256_rcon`. Trong AES-256, nhánh sử dụng Rcon xuất hiện tại những lần sinh word tương ứng với chỉ số bội số của tám trong Key Schedule.

Mạch Cascade XOR

Sau khi tạo được T_r, bốn word mới được tính bằng một chuỗi XOR liên tiếp. Gọi A_0, A_1, A_2, A_3 là bốn word thuộc K_{old}, còn B_0, B_1, B_2, B_3 là những word mới, ta có:

$$\begin{aligned} B_0 &= A_0 \oplus T_r, \\ B_1 &= A_1 \oplus B_0, \\ B_2 &= A_2 \oplus B_1, \\ B_3 &= A_3 \oplus B_2. \end{aligned}$$

Bốn word này được ghép thành một Round Key mới: $K_{\text{new}} = B_0 \parallel B_1 \parallel B_2 \parallel B_3$. Đặc điểm của Cascade XOR là các word phía sau phụ thuộc vào kết quả của word phía trước. Do đó, đường logic sinh B_3 có thể đi qua nhiều tầng XOR hơn so với đường sinh B_0. Đây là một trong những đường tổ hợp cần được xem xét khi phân tích Timing, mặc dù chưa thể kết luận đó là Critical Path nếu chưa có báo cáo STA.

Đồng bộ Key Expansion với Cipher Core

Trong quá trình mã hóa, `round_key_out` được lấy từ K_{recent}. Sau khi Cipher Core hoàn tất cập nhật State cho một Round, cửa sổ khóa đồng thời được cập nhật để cung cấp Round Key cho Round kế tiếp. Ví dụ, sau Initial AddRoundKey, thanh ghi chứa hai nhóm Round Key $K^{(0)}$ và $K^{(1)}$. Tại Round 1, Cipher Core sử dụng $K^{(1)}$, trong khi Key Expansion tạo $K^{(2)}$ để chuẩn bị cho Round 2. Quá trình này tiếp tục cho đến Round cuối cùng. Việc phối hợp cập nhật State Register và Key Register trên cùng cạnh Clock giúp tránh phải lưu riêng toàn bộ Key Schedule. Tuy nhiên, hai module phải sử dụng nhất quán chỉ số Round và tín hiệu điều khiển; nếu Key Expansion cập nhật sai thời điểm, Cipher Core có thể sử dụng Round Key không tương ứng với State đang xử lý.

3.2.4 Thiết kế logic điều khiển Round

Logic điều khiển có nhiệm vụ quản lý quá trình bắt đầu mã hóa, cập nhật State, sinh Round Key và xác định thời điểm hoàn tất. Trong biến thể LUT, phần điều khiển được hiện thực chủ yếu bằng bộ đếm Round và các điều kiện logic, thay vì một FSM khai báo tường minh bằng nhiều trạng thái.

Cụ thể, thiết kế sử dụng tín hiệu `cipher_counter` có độ rộng 4 bit để theo dõi tiến trình mã hóa. Giá trị bộ đếm được đưa ra qua `round_num`, đồng thời được sử dụng để lựa chọn phép biến đổi và Rcon trong Key Expansion.

Điều kiện kết thúc được xác định bằng:

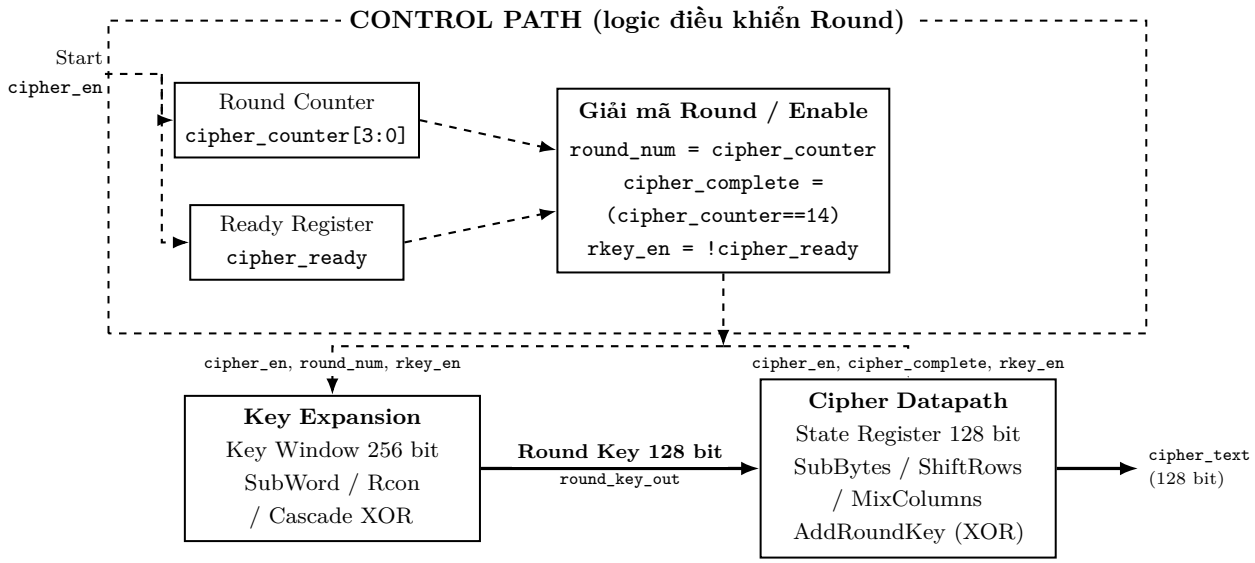
$$\texttt{cipher_complete} = (\texttt{cipher_counter} = 14)\,.$$

Khi nhận `cipher_en`, trạng thái `cipher_ready` được đưa về 0, cho biết lõi đang thực hiện mã hóa. Trong quá trình hoạt động, bộ đếm tiếp tục tăng và State Register được cập nhật qua các Round. Khi bộ đếm đạt 14, logic điều khiển lựa chọn đường dữ liệu bỏ qua MixColumns và cập nhật trạng thái hoàn tất. Tín hiệu cho phép Key Expansion được xác định bởi:

$$\texttt{rkey_en} = \neg\texttt{cipher_ready}.$$

Do đó, trong giai đoạn lõi đang xử lý, tín hiệu `rkey_en` được kích hoạt để Key Expansion cập nhật cửa sổ khóa cùng với tiến trình của Cipher Core.

Mối quan hệ giữa bộ đếm Round, logic tạo tín hiệu điều khiển và các khối xử lý được minh họa trong Hình 5. Sơ đồ chỉ thể hiện đường điều khiển; các phép biến đổi trên dữ liệu State và quá trình sinh Round Key đã được trình bày ở các mục trước.



Nét đứt: tín hiệu điều khiển; nét liền đậm: dữ liệu Round Key / Ciphertext.

Hình 5: Quan hệ giữa logic điều khiển Round, Key Expansion và Cipher Datapath trong AES-256 LUT.

Có thể phân chia hoạt động điều khiển thành các giai đoạn chức năng trong Bảng 7. Các giai đoạn này dùng để diễn giải hành vi của bộ đếm và tín hiệu điều khiển, không phải tên các State được khai báo trực tiếp trong RTL LUT.

Bảng 7: Các giai đoạn điều khiển của lõi AES-256 LUT.

Giai đoạn	Điều kiện	Hoạt động
Chờ lệnh	`cipher_ready=1`	Giữ trạng thái chờ, tiếp nhận yêu cầu Start.
Khởi tạo	`cipher_en=1`	Thực hiện Initial AddRoundKey, nạp Encryption Key và đưa bộ đếm sang Round 1.
Xử lý Round	`round_num=1..13`	Thực hiện SubBytes, ShiftRows, MixColumns, AddRoundKey và cập nhật Key Expansion.
Round cuối	`round_num=14`	Bỏ qua MixColumns, cập nhật Ciphertext và xác lập trạng thái hoàn tất.
Hoàn tất	`cipher_ready=1`	Giữ kết quả trong State Register cho đến khi nhận tác vụ mới.

`cipher_ready` là tín hiệu trạng thái mức logic, không phải xung Done chỉ xuất hiện trong một chu kỳ. Theo RTL, tín hiệu này ở mức 1 khi lõi không xử lý tác vụ và được đưa về 1 khi quá trình mã hóa kết thúc. Vì vậy, khối điều khiển bên ngoài phải phân biệt trạng thái sẵn sàng ban đầu với việc hoàn tất một lệnh mã hóa mới. Ngoài ra, lõi LUT được thiết kế để nhận một xung Start khi đang ở trạng thái chờ. RTL không có một cơ chế hàng đợi lệnh độc lập; vì vậy, việc không phát yêu cầu mã hóa mới trong lúc Core đang xử lý là một điều kiện cần được bảo đảm ở cấp điều khiển hệ thống.

Đối với biến thể BRAM, logic điều khiển có cấu trúc khác do kết quả đọc S-box chỉ sẵn sàng theo xung Clock. RTL BRAM khai báo ba trạng thái `ST_IDLE`, `ST_SB_READ` và `ST_ROUND_UPDATE`. Phân tích chi tiết FSM này sẽ được trình bày trong mục thiết kế biến thể BRAM để làm rõ sự khác biệt về Latency.

3.2.5 Trình tự xử lý và đồng bộ tín hiệu

Trong kiến trúc LUT, Cipher Datapath và Key Expansion hoạt động đồng bộ theo `clk_sys`. Các phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey được mô tả dưới dạng logic tổ hợp; State Register và Key Register nhận dữ liệu mới tại cạnh lên của Clock theo điều kiện cho phép. Khi tín hiệu Start được chấp nhận,

Initial AddRoundKey được thực hiện và kết quả được ghi vào State Register. Tại cùng cạnh Clock, Encryption Key 256 bit được nạp vào `key_reg`, còn bộ đếm được thiết lập để bắt đầu Round 1. Ở các cạnh Clock tiếp theo, State được cập nhật theo kết quả của Round hiện tại. Song song với đó, Key Expansion tính toán và cập nhật cửa sổ khóa, chuẩn bị Round Key cho lần xử lý kế tiếp. Bảng 8 tóm tắt trình tự cập nhật của lõi LUT, với quy ước E_0 là cạnh Clock chấp nhận lệnh Start.

Bảng 8: Trình tự cập nhật State và Round Key của AES-256 LUT.

Cạnh Clock	Giai đoạn	Cập nhật State	Ready sau cạnh
E_0	Khởi tạo	$S_0 = P \oplus K^{(0)}$	0
E_1	Round 1	S_1 sử dụng $K^{(1)}$	0
E_2	Round 2	S_2 sử dụng $K^{(2)}$	0
⋮	⋮	⋮	⋮
E_{13}	Round 13	S_{13} sử dụng $K^{(13)}$	0
E_{14}	Round 14	$S_{14} = C$, sử dụng $K^{(14)}$	1

Theo trình tự trên, lõi LUT thực hiện một lần cập nhật State cho Initial AddRoundKey và 14 lần cập nhật cho các Round mã hóa, tổng cộng 15 lần cập nhật tại các cạnh Clock. Tính từ cạnh nhận Start đến cạnh xác lập Ciphertext cuối cùng, có 14 khoảng chu kỳ Clock giữa hai mốc này. Khi đo Latency, cần nêu rõ quy ước tính chu kỳ và thời điểm lấy mẫu tín hiệu để tránh nhầm lẫn giữa số lần cập nhật State và thời gian hoàn tất tác vụ. Đặc biệt, trong RTL LUT, State Register không có nhánh Reset riêng; dữ liệu thanh ghi được ghi đè khi một lệnh mã hóa được chấp nhận. Vì vậy, giá trị `cipher_text` trước khi thực hiện tác vụ đầu tiên không được xem là Ciphertext hợp lệ. Tương tự, Key Register được khởi tạo bằng Encryption Key tại thời điểm Start. Khi tích hợp với AXI4-Lite Controller, phần mềm không trực tiếp lấy mẫu các giá trị State trung gian. Thay vào đó, Controller sử dụng tín hiệu trạng thái để xác định thời điểm lưu Ciphertext cuối cùng và cung cấp kết quả qua các thanh ghi Memory-Mapped. Cách tổ chức này tách thời gian xử lý bên trong AES Core khỏi thời gian giao tiếp giữa PS và PL.

Những vấn đề cần xem xét về Timing

Do kiến trúc LUT sử dụng Datapath tổ hợp giữa các lần cập nhật State Register, độ trễ của các khối SubBytes, MixColumns, AddRoundKey và mạng định tuyến có thể ảnh hưởng đến chu kỳ Clock nhỏ nhất mà thiết kế đáp ứng được. Đối với Key Expansion, đường tính toán từ S-box qua các tầng Cascade XOR cũng cần được xem xét. Tuy nhiên, chỉ dựa trên cấu trúc RTL chưa thể xác định chắc chắn đường nào là Critical Path, bởi kết quả còn phụ thuộc vào Synthesis, Placement, Routing và các Timing Constraints. Vì vậy, phần thiết kế này chỉ xác định những đường xử lý cần quan tâm. Việc đánh giá Worst Negative Slack (WNS), Maximum Clock Frequency ($F_{\max}$), Latency và Throughput sẽ dựa trên kết quả thực nghiệm trong các chương tiếp theo.

Sự đồng thời của Cipher và Key Expansion. Ở pha khởi tạo, State Register chốt $P \oplus K^{(0)}$ đồng thời Key Register nhận 256 bit khóa gốc. Trong các Round tiếp theo, Cipher dùng Round Key hiện hành để tạo State mới, còn Key Expansion tạo và chốt cửa sổ khóa cho bước sau. Cả hai dùng cùng cạnh Clock nhưng tính toán trên giá trị thanh ghi *trước* cạnh ấy; việc diễn giải theo kiểu "Key cập nhật xong rồi Cipher mới chạy trong cùng cạnh" sẽ sai với ngữ nghĩa nonblocking assignment của SystemVerilog.

Bảng 9: Quan hệ dữ liệu–điều khiển trong lõi LUT ở cấp Register Transfer.

Tín hiệu/thanh ghi	Nguồn điều khiển	Vai trò
`cipherText_reg`	`cipher_en`, `rkey_en`	Chốt State sau Init hoặc sau một Round, xuất Ciphertext cuối
`key_reg[255:0]`	`cipher_en`, `rkey_en`	Chốt khóa gốc hoặc cập nhật cửa sổ tám word
`round_num`	`cipher_counter`	Chỉ số Round dùng chung cho Cipher và Key Expansion
`cipher_complete`	`cipher_counter=14`	Điều khiển bypass MixColumns ở Round cuối
`cipher_ready`	Bộ điều khiển Round	Chỉ thị Core không còn thực hiện giao dịch

Đối với chuỗi cập nhật E_0–E_{14} ở Bảng 8, E_0 là cạnh chấp nhận Start, còn E_{14} là cạnh cập nhật Round cuối. Như vậy, có 15 sự kiện cập nhật State nếu tính cả Initial AddRoundKey, nhưng chỉ 14 khoảng Clock giữa E_0 và E_{14}. Testbench báo độ trễ 15 chu kỳ do cách đặt mốc lấy mẫu và quan sát Ready. Hai cách đếm này không mâu thuẫn nếu ghi rõ định nghĩa.

Ý nghĩa của tín hiệu Ready. `cipher_ready` là trạng thái mức, không phải xung chỉ hợp lệ đúng một Clock. Khi Ready ở mức 1 trước Start, Ciphertext có thể là kết quả cũ. Do đó, AXI Controller phải có điều kiện phân biệt một giao dịch mới đã thực sự trải qua pha Busy trước khi chốt DONE; phiên bản Controller được mô tả ở Mục 3.4 vẫn có giới hạn tại điểm này. Đây là ràng buộc quan trọng khi phân tích tính đúng ở cấp tích hợp, bên cạnh các phép biến đổi AES thuần túy.

3.3 Thiết kế các biến thể phần cứng AES-256

Từ kiến trúc AES-256 Core trình bày ở Mục 3.2, nhóm xây dựng bốn biến thể phần cứng gồm LUT S-box, BRAM S-box, Composite Field và Dummy Noise. Các biến thể cùng nhận khóa mã hóa 256 bit, Plaintext 128 bit và tín hiệu bắt đầu `cipher_en`; kết quả được trả qua `cipher_text[127:0]` và tín hiệu trạng thái `cipher_ready`. Sự khác biệt chủ yếu nằm ở phương pháp hiện thực S-box, cách tổ chức chu kỳ xử lý hoặc sự bổ sung logic hoạt động song song với lõi mã hóa. Việc duy trì giao diện chung tạo điều kiện sử dụng cùng một lớp điều khiển ở các kiến trúc DPR và Replica, sẽ được trình bày trong Mục 3.5.

Nội dung sau tập trung vào những thay đổi được thể hiện trong RTL của từng biến thể. Các phép biến đổi AES cơ bản và cơ chế Rolling Datapath đã được giải thích ở Chương 2 và Mục 3.2 nên không trình bày lại. Tương tự, thông số tài nguyên, thời gian thực thi đo trên KV260 và kết quả kiểm chứng được dành cho Chương 5 và Chương 6.

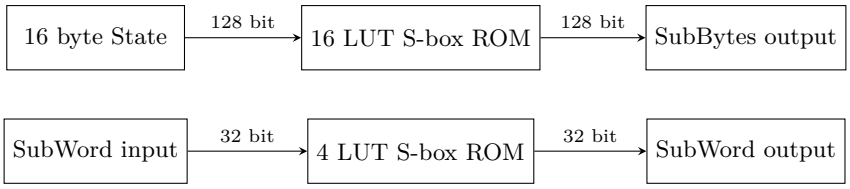
3.3.1 Biến thể LUT S-box

Biến thể LUT được hiện thực trong tệp `aes256_lut.sv`, gồm hai khối chính `aes256_cipher_core` và `aes256_key_expansion`.

Đây là biến thể tham chiếu khi so sánh với các phương án còn lại. Điểm đặc trưng của thiết kế là S-box được mô tả bằng bảng tra 256 phần tử, mỗi phần tử rộng 8 bit, với đầu ra tổ hợp. Module `aes_sbox_lut_rom` sử dụng mảng `sbox_rom[0:255]` và phép gán `dout = sbox_rom[addr]`.

Thuộc tính `rom_style = "distributed"` được đặt trên mảng ROM để hướng công cụ Vivado suy diễn bảng tra trên tài nguyên LUT.

Trong Cipher Datapath, 128 bit State được chia thành 16 byte và nối tới 16 thể hiện S-box hoạt động song song. Mỗi byte địa chỉ tạo ra một byte sau SubBytes; các đầu ra được ghép thành bus `after_subBytes[127:0]`. Key Expansion sử dụng thêm bốn S-box cùng loại để tính `after_subW[31:0]`. Như vậy, trong mô tả RTL tồn tại 20 vị trí tra cứu S-box gồm 16 vị trí cho State và bốn vị trí cho khóa. Con số này phản ánh số thể hiện logic ở mức RTL, không mặc nhiên bằng số lượng khối nhớ vật lý sau tổng hợp.



Hình 6: Tổ chức các S-box dạng LUT trong Cipher Datapath và Key Expansion.

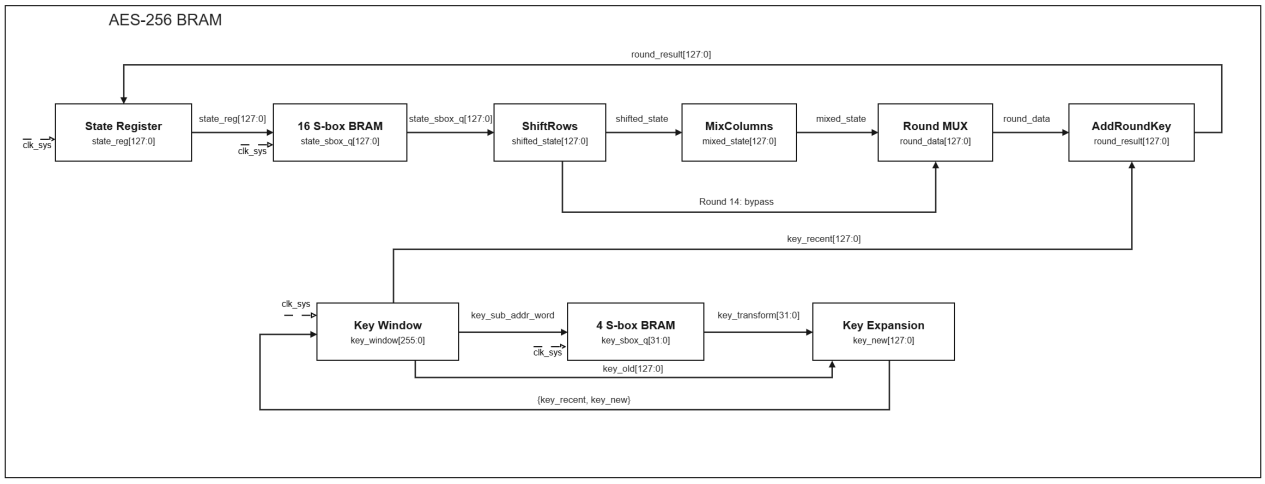
Việc tra cứu theo logic tổ hợp cho phép đầu ra S-box phụ thuộc trực tiếp vào byte địa chỉ trong cùng chu kỳ, không cần trạng thái chờ đọc bộ nhớ. Điều này phù hợp với cơ chế cập nhật State theo từng Round của biến thể LUT ở Mục 3.2. Tuy nhiên, đường truyền dữ liệu của một Round vẫn phải đi qua S-box và các phép biến đổi phía sau trước khi được chốt tại cạnh Clock; do đó, thời gian đường tổ hợp và số LUT thực tế cần được xác định bằng kết quả tổng hợp và phân tích Timing, thay vì suy ra chỉ từ cách mô tả ROM.

3.3.2 Biến thể BRAM S-box

Biến thể BRAM trong `aes256_bram.sv` thay đổi cả cách truy xuất S-box lẫn trình tự điều khiển. Module `aes_sbox_bram` vẫn lưu bảng thay thế 256 byte nhưng đầu ra được cập nhật tại cạnh lên của Clock thông qua phép gán `q <= rom[addr]`. Như vậy, dữ liệu đọc không còn được sử dụng tức thời theo địa chỉ như ở biến thể LUT. Thiết kế phải dành một pha để ghi nhận kết quả tra cứu trước khi cập nhật State.

Trong Cipher Core có 16 S-box BRAM hoạt động song song. Chúng nhận các byte từ `state_reg[127:0]` và tạo bus `state_sbox_q[127:0]`.

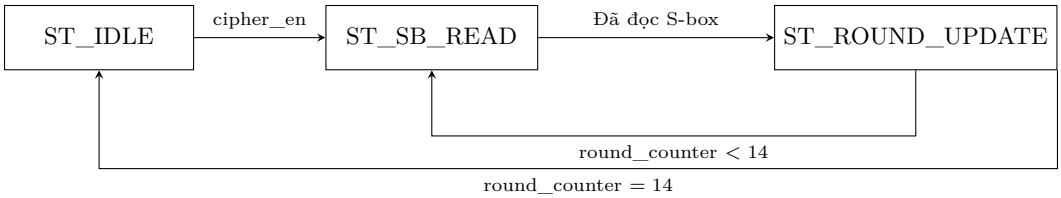
Bốn thể hiện khác phục vụ Key Expansion, với đầu vào 32 bit `key_sub_addr_word` và đầu ra `key_sbox_q[31:0]`. Hai nhóm tra cứu dùng chung Clock `clk_sys` nên kết quả S-box của State và Key Expansion được ghi nhận trong cùng pha đọc. Các phép ShiftRows, MixColumns, chọn nhánh Round cuối và AddRoundKey tiếp tục được triển khai bằng logic tổ hợp sau đầu ra S-box đã chốt.



Hình 7: Cấu trúc dữ liệu của biến thể BRAM, gồm các S-box đọc đồng bộ cho State và Key Expansion.

Khác với biến thể LUT, `aes256_bram` tổ chức State và cửa sổ khóa ngay trong cùng một module, lần lượt qua các thanh ghi `state_reg[127:0]` và `key_window[255:0]`. Khi nhận `cipher_en` ở trạng thái nghỉ, module thực hiện Initial AddRoundKey bằng `plain_text` $\oplus$ `cipher_key[255:128]`, đồng thời nạp khóa 256 bit vào `key_window`. Trong các Round tiếp theo, `key_recent = key_window[127:0]` được sử dụng làm Round Key, còn `key_new` được tính bằng chuỗi XOR từ `key_old` và `key_transform`. Cửa sổ khóa chỉ dịch sau khi hoàn thành một Round chưa phải Round cuối, tránh cập nhật khóa trong pha truy xuất S-box.

Để đồng bộ các thao tác trên, thiết kế dùng FSM ba trạng thái `ST_IDLE`, `ST_SB_READ` và `ST_ROUND_UPDATE`. Sơ đồ chuyển trạng thái ở Hình 8 được rút trực tiếp từ khối điều khiển trong RTL.



Hình 8: Trình tự điều khiển ba trạng thái của biến thể BRAM.

Ở `ST_IDLE`, bộ điều khiển chờ lệnh Start và nạp State, khóa ban đầu. Tại `ST_SB_READ`, các S-box nhận địa chỉ từ thanh ghi hiện tại; dữ liệu đọc được ghi vào đầu ra `q` tại cạnh Clock. Đến `ST_ROUND_UPDATE`, logic sử dụng kết quả vừa đọc để tính `round_result`, ghi lại `state_reg` và, nếu chưa đến Round 14, cập nhật `key_window` cùng bộ đếm Round. Khi xử lý Round 14, `round_result` được ghi vào State, `cipher_ready` được đưa lên mức 1 và FSM trở lại `ST_IDLE`. Điều kiện chọn dữ liệu Round cuối được thể hiện bằng:

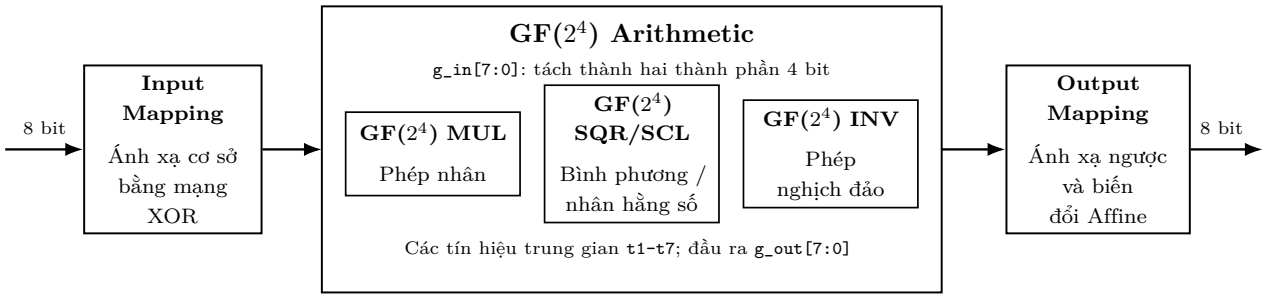
$$\texttt{round_data} = \begin{cases} \texttt{shifted_state}, & \texttt{round_counter} = 14, \\ \texttt{mixed_state}, & \texttt{round_counter} \neq 14. \end{cases}$$

Cơ chế đọc đồng bộ làm tăng số pha xử lý so với LUT, nhưng tách việc đọc S-box khỏi bước cập nhật Round, qua đó thay đổi cấu trúc đường Timing. Một điểm cần phân biệt là `aes_sbox_bram` hiện sử dụng thuộc tính `ramstyle = "M10K"` trong RTL. Thuộc tính này không phải chỉ thị `ram_style = "block"` thường dùng của Vivado; vì vậy không thể chỉ dựa vào tên biến thể hoặc thuộc tính RTL để kết luận các bảng tra đã được ánh xạ thành BRAM vật lý trên KV260. Việc xác nhận phải căn cứ vào báo cáo tổng hợp và sử dụng tài nguyên FPGA ở Chương 5.

3.3.3 Biến thể Composite Field

Biến thể Composite Field được mô tả trong tệp `aes_composite.sv`. Khác với hai phương án sử dụng bảng tra, S-box của biến thể này được hiện thực bằng mạng logic tổ hợp dựa trên các phép toán của Composite Field. Lõi mã hóa vẫn giữ 128 bit State, cấu trúc Round và cách truyền Round Key tương tự biến thể LUT; phần được thay thế trực tiếp là module `aes_sbox` trong Cipher Core và Key Expansion. Theo RTL hiện tại, Cipher Core tạo 16 thể hiện `aes_sbox` cho SubBytes; Key Expansion gọi cùng loại S-box bốn lần để xử lý `after_rotW[31:0]`.

Cấu trúc bên trong mỗi S-box chia thành ba bước xử lý. Trước tiên, tám bit `in_byte[7:0]` được biến đổi qua các tổ hợp XOR để tạo `g_in[7:0]`, tương ứng với ánh xạ từ biểu diễn byte AES sang biểu diễn Composite Field. Tiếp theo, `g_in` được tách thành hai nửa bốn bit và xử lý thông qua các tín hiệu trung gian `t1` đến `t7`. Cuối cùng, hai kết quả bốn bit được ghép thành `g_out[7:0]`; mạng XOR/XNOR đầu ra thực hiện ánh xạ ngược kết hợp biến đổi Affine để tạo `out_byte[7:0]`.



Hình 9: Các khối chức năng trong một S-box Composite Field.

Phần số học trường con được cấu tạo theo phân cấp. Module `GF_2_4_MUL_Nw_PolyPoly` thực hiện phép nhân trên trường bốn bit; `GF_2_4_INV_Nw_PolyPoly` thực hiện phép nghịch đảo; còn `GF_2_4_SQR_SCL_N_N2_Nw_PolyPoly` đảm nhiệm phép bình phương kết hợp nhân với hằng số. Các module này tiếp tục sử dụng những khối xử lý hai bit như `GF_2_2_MUL_Poly`, `GF_2_2_INV_Poly` và các phép biến đổi tuyến tính khác. Như vậy, thao tác thay thế byte được biểu diễn bằng những mạng XOR, AND và các tổ hợp trường hữu hạn thay vì truy xuất một mảng ROM. Cơ sở toán học của cách phân rã trường đã được trình bày ở Chương 2; tại đây, vấn đề quan tâm là quan hệ giữa các module trong RTL.

Toàn bộ module `aes_sbox` là logic tổ hợp, không chứa thanh ghi Clock bên trong. Vì vậy, biến thể Composite không cần bổ sung pha đọc S-box đồng bộ như BRAM và vẫn sử dụng cơ chế điều khiển Round tương tự biến thể LUT. Tuy nhiên, thời gian của đường tổ hợp có thể chịu ảnh hưởng từ các tầng nhân, nghịch đảo trường con và mạng XOR ở đầu ra. Không nên mặc định Composite Field sẽ nhanh hơn hay ít tốn LUT hơn phương án bảng tra: điều này phụ thuộc cách công cụ tổng hợp tối ưu cấu trúc logic trên thiết bị mục tiêu. Đánh giá định lượng được thực hiện ở phần kết quả triển khai.

3.3.4 Biến thể Dummy Noise

Biến thể Dummy Noise trong `dummy_noise.sv` không thay đổi các phép biến đổi AES chính mà bổ sung một nhánh logic có hoạt động chuyển mạch trong lúc lõi mã hóa vận hành. Wrapper `dummy_noise` vẫn kết nối Cipher Core và Key Expansion theo giao diện thông thường. Điểm khác biệt nằm trong `aes256_cipher_core_cpng`, nơi có thêm module `aes256_full_cpng` với các thanh ghi và S-box riêng. Thiết kế này được sử dụng để nghiên cứu ảnh hưởng của hoạt động logic bổ sung đối với đặc tính công suất. Nhánh CPNG nhận dữ liệu State 128 bit từ Cipher Core. Cổng đầu vào này có tên `state_before_addroundkey[127:0]`.

Dữ liệu được nối từ tín hiệu `addRoundKey_in[127:0]` của lõi AES.

Thanh ghi `kinterf_reg[127:0]` được cập nhật theo hàm LFSR. Ở mỗi cạnh lên khi `update_en` được kích hoạt, các thanh ghi được cập nhật theo các biểu thức dưới đây, trong đó C là `cpng_state_reg`, K là `kinterf_reg` và S_{preARK} là `state_before_addroundkey`:

$$C^{+} = S_{\text{preARK}} \oplus K, \tag{8}$$

$$K^{+} = \text{LFSR128}(K). \tag{9}$$

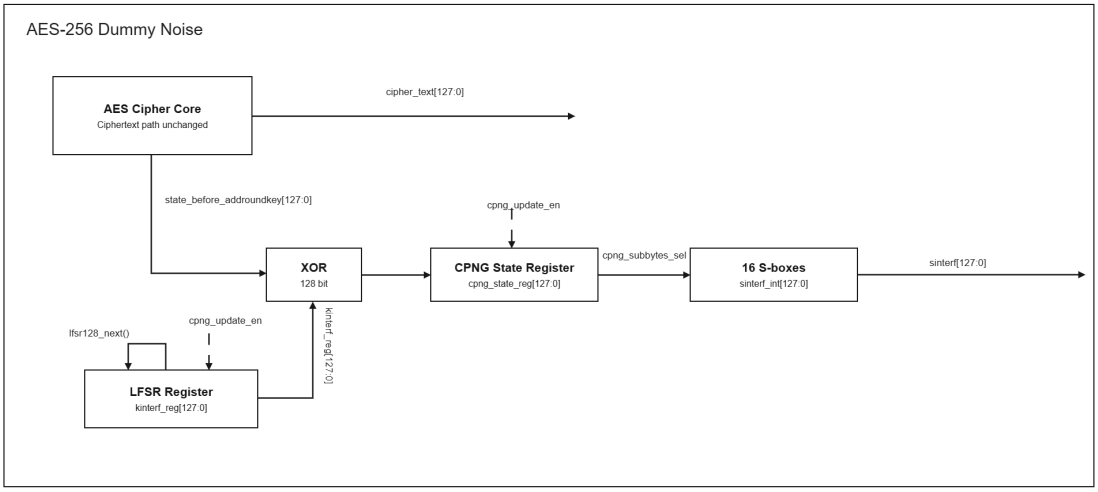
Trong đó, phép cập nhật LFSR dịch chuỗi 128 bit sang trái một vị trí và tính bit phản hồi từ bốn vị trí của trạng thái cũ:

$$f = k[127] \oplus k[125] \oplus k[100] \oplus k[98], \qquad k^{+} = \{k[126{:}0], f\}.$$

Ở trạng thái Reset, `cpng_state_reg` được đưa về 0, trong khi `kinterf_reg` được nạp giá trị `SEED` cố định khai báo trong module. Điều này có nghĩa chuỗi trạng thái LFSR là xác định nếu cùng điều kiện khởi tạo và Enable; nguồn này không phải bộ sinh số ngẫu nhiên vật lý. Tín hiệu cho phép cập nhật được tạo từ trạng thái hoạt động của AES:

$$\texttt{cpng_update_en} = \texttt{cipher_en} \vee \big(\texttt{rkey_en} \wedge \neg \texttt{cipher_complete}\big).$$

Sau khi được chốt, `cpng_state_reg` đi qua mạng hoán vị byte `cpng_subbytes_sel` và 16 lần gọi hàm S-box, tạo `sinterf_int[127:0]`. Đầu ra này được nối tới `sinterf` của CPNG và `cpng_sinterf` trong Cipher Core. Trong RTL được khảo sát, `cpng_sinterf` không tham gia phép tính `after_addRoundkey` và không được nối vào bus Ciphertext. Vì vậy, nhánh CPNG về mặt chức năng được tách khỏi đường dữ liệu tạo kết quả mã hóa; sự hiện diện của nó hướng tới tạo thêm chuyển mạch nội bộ chứ không làm biến đổi thuật toán AES.



Hình 10: Nhánh CPNG hoạt động song song với Cipher Core trong biến thể Dummy Noise.

Trong RTL, một số thanh ghi và module CPNG có sử dụng các thuộc tính như `KEEP`, `DONT_TOUCH` và `KEEP_HIERARCHY` nhằm hạn chế công cụ loại bỏ hoặc tái cấu trúc nhánh logic không ảnh hưởng đến đầu ra AES. Tuy nhiên, mức độ bảo toàn logic thực tế vẫn cần được kiểm tra trên netlist sau tổng hợp và Implementation. Tương tự, việc có thêm các nút chuyển mạch không đủ để kết luận mức tiêu thụ công suất đã tăng theo một tỷ lệ nhất định hoặc khả năng chống phân tích công suất đã được cải thiện. Các kết luận này chỉ có thể đưa ra sau khi đo hoặc mô phỏng công suất theo cùng điều kiện giữa các biến thể.

Lịch hoạt động của BRAM theo các pha Clock. Hai nhóm S-box BRAM gồm 16 đường tra State và bốn đường tra SubWord cùng nhận Clock lõi. Trong một lượt `ST_SB_READ`, chúng lấy mẫu địa chỉ từ các thanh ghi State/Key đang ổn định. Ở lượt `ST_ROUND_UPDATE` kế tiếp, dữ liệu S-box đã được chốt mới được dùng để tính ShiftRows, MixColumns (hoặc bypass Round 14) và AddRoundKey trước khi State Register được cập nhật. Cách sắp xếp này bảo đảm Key Expansion và Cipher không dùng kết quả đọc ROM của hai Round khác nhau.

Bảng 10: Trình tự xử lý trong một Round của biến thể BRAM.

Pha FSM	**Thanh ghi/đường dữ liệu**	**Hành vi tại cạnh Clock**
ST_IDLE	State, Key Window	Khi Start: chốt Initial AddRoundKey và khóa gốc, khởi tạo bộ đếm
ST_SB_READ	S-box State và S-box SubWord	Cập nhật các đầu ra ROM đồng bộ theo địa chỉ State/Key của Round hiện hành
ST_ROUND_UPDATE	State, Key Window, Round Counter	Chốt State mới; cập nhật khóa nếu chưa đến Round cuối, hoặc đưa Ready lên 1

Vì mỗi Round cần pha đọc và pha cập nhật, số chu kỳ điều khiển lớn hơn LUT. Với 14 Round, mô phỏng ghi nhận 29 chu kỳ ở BRAM và 15 chu kỳ ở LUT theo Testbench. Đây là hệ quả của lịch RTL; các giá trị không được diễn giải thành thời gian truy cập BRAM vật lý riêng lẻ. Tại Round 14, FSM vẫn thực hiện pha đọc S-box nhưng MUX bỏ qua MixColumns trước AddRoundKey.

Mức độ sử dụng lại phần cứng theo thời gian. LUT bố trí 16 S-box cho SubBytes và bốn S-box cho Key Expansion: chúng hoạt động song song ở cấp RTL khi cùng Round cần xử lý. BRAM cũng mô tả 16 đường tra State và bốn đường tra khóa nhưng yêu cầu đầu ra đồng bộ, được ánh xạ thành tài nguyên RAM vật lý tùy kết

quả Synthesis. Composite thay mỗi vị trí S-box bằng mạng tính toán tổ hợp và giữ lịch Round tương tự LUT. Dummy Noise bổ sung nhánh CPNG có Register/Enable riêng, không thay vị trí phép AddRoundKey trong Cipher Datapath.

3.3.5 So sánh cấu trúc các biến thể

Bốn biến thể có thể được phân biệt theo phương pháp thực hiện S-box, trạng thái lưu trữ và logic điều khiển bổ sung. Bảng 11 tổng hợp các đặc điểm được xác nhận từ RTL, chưa bao gồm kết quả đo tài nguyên hay hiệu năng sau khi đưa lên FPGA.

Bảng 11: Đối chiếu cấu trúc RTL của bốn biến thể AES-256.

Tiêu chí	LUT	BRAM	Composite Field	Dummy Noise
S-box chính	ROM tổ hợp, hướng đến distributed LUT	ROM đọc đồng bộ theo Clock	Mạng số học Composite Field	Tra bảng bằng hàm tổ hợp như lõi nền
S-box ở mức RTL	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa; thêm 16 S-box CPNG
Điều khiển Round	Bộ đếm Round	FSM ba trạng thái và bộ đếm Round	Bộ đếm Round	Bộ đếm Round; thêm Enable cho CPNG
Đường dữ liệu bổ sung	Không	Đường đọc S-box có chốt đầu ra	Logic trường con thay ROM	LFSR, thanh ghi CPNG và mạng S-box riêng
Đặc điểm cần đánh giá	LUT và Timing đường tổ hợp	Memory mapping, số chu kỳ, Timing	LUT, chiều sâu logic và Timing	Tài nguyên logic, công suất và switching activity

Số chu kỳ xử lý ở mức RTL. Để đối chiếu lịch điều khiển giữa bốn biến thể, các Testbench độc lập áp dụng cùng một vector AES-256 Known Answer Test, Clock 10 ns và cách đếm chu kỳ từ xung `cipher_en` đến khi `cipher_ready` báo hoàn tất. Kết quả mô phỏng ghi nhận LUT, Composite Field và Dummy Noise cùng mất 15 chu kỳ theo bộ đếm Testbench, trong khi BRAM mất 29 chu kỳ. Mỗi biến thể đều trả về Ciphertext `8EA2B7CA516745BFEAFC49904B496089`, khớp giá trị tham chiếu. Độ trễ BRAM lớn hơn vì các Round có thêm pha đọc S-box đồng bộ; nhánh CPNG chạy song song nên không làm tăng số chu kỳ điều khiển của đường mã hóa trong thử nghiệm này.

Các con số trên phản ánh *quy ước đếm của Testbench* tại 100 MHz, không phải số chu kỳ được đo bằng bộ đếm phần cứng trên KV260 hay thời gian giao dịch AXI/Linux. Kết quả chứng minh tính đúng đắn đối với vector thử nghiệm đã dùng; kiểm chứng NIST nhiều vector được trình bày riêng ở Chương 5.

Biến thể LUT có ưu điểm về cấu trúc điều khiển đơn giản và phù hợp làm mốc tham chiếu. BRAM thay đổi cách truy xuất S-box, đồng thời cần FSM để bảo đảm dữ liệu đọc được sử dụng đúng thời điểm. Composite Field giữ tiến trình xử lý theo Round nhưng thay bảng tra bằng mạng số học trường con. Dummy Noise lại giữ đường dữ liệu AES và bổ sung một nhánh CPNG hoạt động song song. Nhờ đó, phép so sánh giữa các biến thể không chỉ phản ánh sự thay đổi một module S-box, mà còn cho thấy ảnh hưởng của lựa chọn kiến trúc đến cách tổ chức dữ liệu và điều khiển.

Sơ đồ sử dụng S-box và lưu dữ liệu trung gian. Bốn biến thể cần được so sánh theo hai trục: số phần tử hoạt động song song *trong một pha* và số lần tái sử dụng cùng phần tử *qua các Round*. Trong LUT, 16 S-box Cipher và bốn S-box SubWord là hai nhóm logic riêng. Cùng một nhóm Cipher được dùng lại lần lượt cho các Round 1–14, thay vì tạo 14 nhóm SubBytes độc lập. Với BRAM, hai nhóm cũng tồn tại ở cấp RTL nhưng kết quả đọc chỉ hợp lệ sau pha đồng bộ. Composite giữ cùng cách tổ chức số đường xử lý, chỉ thay khối thực hiện phép ánh xạ byte. Noise thêm nhánh CPNG được cập nhật theo Enable; Ciphertext không lấy dữ liệu từ nhánh này.

Bảng 12: Quan hệ giữa số đường xử lý và thời điểm chốt dữ liệu.

Biến thể	Trong một Round	Qua các Round kế tiếp
LUT	16 Cipher S-box và 4 SubWord S-box tổ hợp	State/Key cập nhật ở cạnh Clock; tái sử dụng các S-box ở Round kế
BRAM	16 + 4 đường truy xuất ROM đồng bộ	Pha READ chốt đầu ra S-box; pha UPDATE chốt State và Key Window
Composite	16 + 4 S-box logic trường con	Không có thanh ghi riêng trong S-box; hồi tiếp State theo Round
Noise	S-box Cipher/Key và nhánh CPNG độc lập	CPNG có thanh ghi cập nhật theo `update_en`, không ghi lại State AES

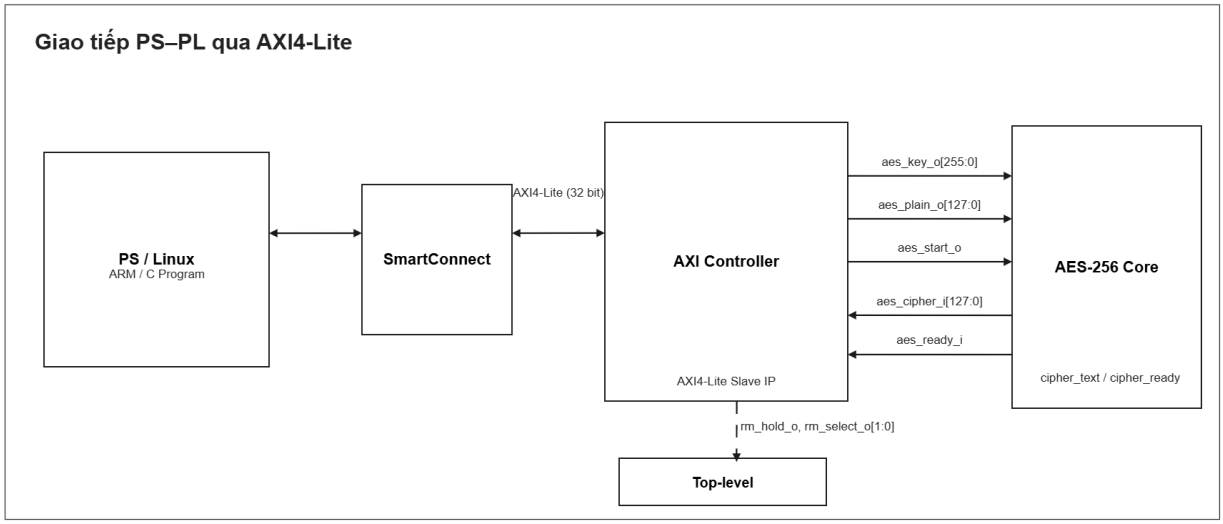
Đường tới hạn và phân chia chu kỳ. Với LUT/Composite, đường tổ hợp của một Round có thể đi qua mạng S-box, hoán vị ShiftRows, logic MixColumns, MUX Round cuối và XOR Key trước khi State được chốt. Key Expansion đồng thời có đường SubWord–Cascade XOR riêng. Biểu thức Setup Timing không đơn thuần là số Round nhân với thời gian một phép toán, vì các Round chạy tại các chu kỳ khác nhau. BRAM tạo một ranh giới thanh ghi tại đầu ra S-box, giúp chia đường xử lý thành pha truy xuất và pha cập nhật, nhưng thêm chu kỳ điều khiển. Việc đường nào thực sự giới hạn Clock phải được xác nhận từ báo cáo Timing sau Routing, không chỉ từ cấu trúc sơ đồ.

3.4 Thiết kế giao tiếp PS–PL

Giao tiếp giữa Processing System (PS) và Programmable Logic (PL) được xây dựng để phần mềm trên ARM có thể nạp khóa, cung cấp Plaintext, khởi động lõi AES-256 và đọc Ciphertext mà không cần thao tác trực tiếp với các tín hiệu nội bộ của Cipher Core. Trong thiết kế của nhóm, lớp giao tiếp này là một AXI4-Lite Slave IP tùy chỉnh, với dữ liệu thanh ghi rộng 32 bit. Mục 3.4 tập trung vào giao diện và ánh xạ thanh ghi; cách kết nối cụ thể với Reconfigurable Partition và các Replica được trình bày ở Mục 3.5.

3.4.1 Kiến trúc kết nối PS–PL

Trên nền tảng KV260, chương trình C chạy dưới Linux ở PS thực hiện các giao dịch đọc/ghi tại vùng địa chỉ AXI được ánh xạ cho bộ điều khiển. Đường truy cập đi qua khối kết nối AXI trong PL đến Custom AXI Controller. Từ các thanh ghi 32 bit, bộ điều khiển tạo hai bus song song `aes_key_o[255:0]` và `aes_plain_o[127:0]`, đồng thời phát xung `aes_start_o` khi có lệnh bắt đầu. Chiều ngược lại, lõi mã hóa cung cấp `aes_cipher_i[127:0]` cùng `aes_ready_i` để bộ điều khiển lưu kết quả và đưa trạng thái về PS.



Hình 11: Kết nối dữ liệu và điều khiển giữa PS, AXI4-Lite Controller và AES-256 Core.

Trong Vivado Block Design, hệ thống sử dụng các khối `zynq_ultra_ps_e_0`, `smartconnect_0` và `proc_sys_reset_0`. Custom AXI Controller được kết nối ở phía Slave của mạng AXI. Tầng này chỉ thực hiện chức năng giao tiếp, không chứa các phép biến đổi AES. Xung Clock và Reset được phân phối từ hệ thống PS–PL để đồng bộ việc truy cập thanh ghi với phần cứng. Đối với mô hình DPR, các tín hiệu từ AXI Controller còn đi qua logic điều khiển và cách ly RM; đối với Replica, tín hiệu chọn biến thể được sử dụng ở tầng Top-level. Những khác biệt này không làm thay đổi quy ước truy cập thanh ghi của phần mềm.

3.4.2 Custom AXI4-Lite Controller

Module `aes256_axi_controller_slave_lite_v1_0_S00_AXI` có bus dữ liệu AXI rộng 32 bit và bus địa chỉ rộng 7 bit. Phần giao thức có các kênh ghi địa chỉ (AW), ghi dữ liệu (W), phản hồi ghi (B), đọc địa chỉ (AR) và đọc dữ liệu (R) theo AXI4-Lite [8]. Bộ điều khiển tiếp nhận giao dịch ghi khi địa chỉ và dữ liệu hợp lệ đồng thời với tín hiệu sẵn sàng tương ứng; phản hồi ghi dùng mã `OKAY`. Với thao tác đọc, địa chỉ được chốt tại kênh AR, dữ liệu 32 bit được chọn từ Register Map và trả qua kênh R. Các tín hiệu `WSTRB[3:0]` cho phép ghi theo từng byte, tránh thay đổi các byte không được chọn khi cập nhật một thanh ghi dữ liệu.

Về chức năng, RTL gồm ba phần: giải mã giao dịch AXI, nhóm thanh ghi điều khiển và dữ liệu, và logic chuyển đổi giữa thanh ghi 32 bit với các cổng dữ liệu AES. Các từ khóa được ghép theo thứ tự `{KEY7, ..., KEY0}` để tạo khóa 256 bit; tương tự, bốn từ Plaintext tạo thành bus 128 bit. Chiều đọc sử dụng thanh ghi `cipher_latched[127:0]` để lưu kết quả mã hóa và tách thành bốn từ 32 bit. Nhờ đó, giao diện của lõi AES không phải thay đổi khi chuyển giữa bốn biến thể phần cứng.

Lệnh Start được hiện thực như một xung điều khiển, không phải một bit lưu liên tục trong thanh ghi. Khi có giao dịch ghi vào `COMMAND` với bit 0 của dữ liệu bằng 1 và `WSTRB[0]` được bật, RTL đưa `aes_start_o` lên mức 1 trong một chu kỳ `S_AXI_ACLK`; tại các chu kỳ còn lại tín hiệu này mặc định trở về 0. Cùng thao tác Start, cờ `done_latched` của yêu cầu trước được xóa. Việc phát xung một chu kỳ giúp Cipher Core nhận một yêu cầu mới mà không phụ thuộc thời gian chương trình giữ dữ liệu ghi trên bus AXI.

3.4.3 Register Map và tín hiệu điều khiển

Register Map được tổ chức theo các offset 4 byte, phù hợp với dữ liệu AXI4-Lite rộng 32 bit. Trong hệ thống thí nghiệm, vùng thanh ghi được ánh xạ tại địa chỉ cơ sở `0xA0000000`; đây là địa chỉ cấu hình của hệ thống tích

hợp chứ không phải hằng số được cố định bên trong RTL của Slave IP. Bảng 13 trình bày các vùng địa chỉ và chiều truy cập tương ứng.

Bảng 13: Ánh xạ thanh ghi của AXI4-Lite Controller.

Offset	Thanh ghi	R/W	Chức năng
0x00	COMMAND	W	Bit 0 phát xung START; đọc về 0.
0x04	STATUS	R	Bit 0: DONE (chốt); bit 1: AES_READY trực tiếp.
0x08	RM_CONTROL	R/W	Bit 0: RM_HOLD; bit [2:1]: RM_SELECT.
0x0C	RESERVED	–	Chưa sử dụng.
0x10-0x2C	KEY0–KEY7	R/W	Tám từ 32 bit tạo aes_key_o[255:0].
0x30-0x3C	PLAIN0–PLAIN3	R/W	Bốn từ 32 bit tạo aes_plain_o[127:0].
0x40-0x4C	CIPHER0–CIPHER3	R	Bốn từ 32 bit từ cipher_latched[127:0].

Với nhóm KEY, địa chỉ 0x10 chứa cipher_key[31:0] và địa chỉ 0x2C chứa cipher_key[255:224]; các từ giữa tiếp tục tăng theo từng 32 bit. Quy ước tương tự được áp dụng với Plaintext và Ciphertext. Chẳng hạn, CIPHER0 đọc phần thấp cipher_latched[31:0], còn CIPHER3 đọc phần cao cipher_latched[127:96]. Sự thống nhất thứ tự từ giữa phần mềm và RTL là điều kiện quan trọng để kết quả trên KV260 trùng với vector kiểm chứng AES.

Trường RM_CONTROL[0] được đưa ra cổng rm_hold_o, sử dụng khi cần giữ hoặc cách ly Reconfigurable Module trong kiến trúc DPR. Hai bit RM_CONTROL[2:1] được đưa ra rm_select_o[1:0] với quy ước 00: LUT, 01: BRAM, 10: Composite Field và 11: Dummy Noise. Việc giải thích cơ chế chọn RM và chuyển đổi cấu hình thuộc Mục 3.5; trong phạm vi 3.4, các bit này được xem là giao diện điều khiển xuất phát từ Register Map.

3.4.4 Handshake và trình tự trao đổi dữ liệu

Một yêu cầu mã hóa được thực hiện qua các bước truy cập thanh ghi. Trước tiên, phần mềm ghi tám từ KEY và bốn từ Plaintext theo đúng thứ tự địa chỉ. Nếu sử dụng Replica, phần mềm chọn biến thể qua RM_SELECT; nếu sử dụng DPR, việc giữ RM trong quá trình đổi cấu hình được quản lý riêng và phải được giải phóng trước khi bắt đầu mã hóa. Sau đó, ghi giá trị có bit 0 bằng 1 vào COMMAND để phát Start. Lõi AES nhận dữ liệu đã nạp, tiến hành mã hóa và cập nhật tín hiệu hoàn tất khi kết quả sẵn sàng. Cuối cùng, phần mềm đọc STATUS, lấy bốn thanh ghi Ciphertext và ghép chúng lại thành kết quả 128 bit. Việc đối chiếu Ciphertext với vector tham chiếu và lưu thời gian thực thi thuộc Chương 5–6.

Ở chiều phản hồi, AXI Controller sử dụng cặp tín hiệu aes_ready_i và aes_cipher_i[127:0]. RTL hiện tại có điều kiện if (aes_ready_i) để chốt Ciphertext và đưa done_latched lên mức 1. Vì vậy, bit STATUS[0] phản ánh cờ DONE đã lưu, trong khi STATUS[1] phản ánh trực tiếp mức tín hiệu aes_ready_i. Hai bit có ý nghĩa khác nhau và không nên sử dụng thay thế cho nhau khi phân tích waveform.

aes_ready_i ở một số biến thể có thể duy trì mức 1 trong trạng thái nghỉ. Trong khi đó, điều kiện chốt DONE của phiên bản AXI Controller đang kiểm tra mức cao, chưa kiểm tra cạnh lên mới của READY sau khi phát Start. Do đó, nếu DONE được xóa tại thời điểm Start nhưng READY vẫn đang cao trong một chu kỳ tiếp theo, cờ này có khả năng được đặt lại trước khi phép mã hóa mới hoàn tất. Đây là điểm cần được kiểm chứng ở mức tích hợp; một hướng hoàn thiện là chỉ cho phép chốt kết quả sau khi đã ghi nhận pha Busy (READY xuống 0) của yêu cầu hiện tại. Phần mô tả này phản ánh đúng hành vi RTL hiện có, không giả định cơ chế cải tiến đã được tích hợp.

Như vậy, AXI4-Lite Controller đóng vai trò giao diện ổn định giữa phần mềm và các lõi AES-256. Các khác biệt

giữa DPR và Replica tập trung ở tầng điều khiển và liên kết phần cứng phía sau giao diện này, trong khi phần mềm vẫn sử dụng các nhóm thanh ghi KEY, PLAIN, COMMAND, STATUS và CIPHER theo cùng quy ước.

Trình tự truyền dữ liệu và khởi động AES Core. Trước khi bắt đầu mã hóa, chương trình ARM ghi khóa 256 bit vào tám thanh ghi KEY0–KEY7 và bản rõ 128 bit vào bốn thanh ghi PT0–PT3 thông qua giao tiếp AXI4-Lite. Mỗi giao dịch ghi truyền một word dữ liệu 32 bit, với các kênh AW, W và B lần lượt đảm nhiệm địa chỉ, dữ liệu và phản hồi ghi.

Sau khi dữ liệu đầu vào được thiết lập, phần mềm ghi giá trị START vào thanh ghi COMMAND. AXI Controller giải mã giao dịch ghi hợp lệ và tạo xung `aes_start_o` để kích hoạt AES Core.

Trong quá trình thực hiện 14 Round, KEY và Plaintext được lưu tại các thanh ghi giao tiếp, thay vì được truyền lại từ ARM sau mỗi Round. Khi mã hóa hoàn tất, phần mềm kiểm tra thanh ghi STATUS và đọc Ciphertext từ bốn thanh ghi CT0–CT3.

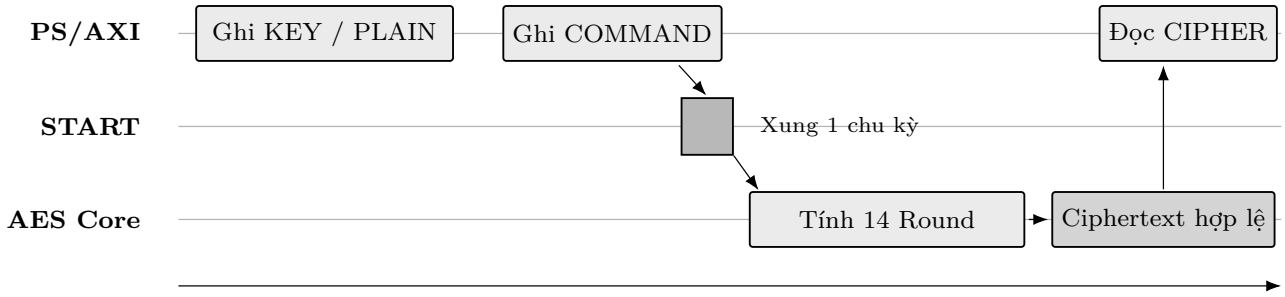
Bảng 14: Các sự kiện quan sát tại ranh giới PS–AXI–AES.

Thời điểm	**AXI/Register Map**	**AES Core**
Tải dữ liệu	Ghi KEY0–7, PLAIN0–3	Chưa phát Start; dữ liệu được lưu tại Slave
Ghi COMMAND	Giải mã `0x00`, tạo xung `aes_start_o`	Nhận Key và Plaintext từ bus song song 256/128 bit
Pha Busy	STATUS theo dõi READY; DONE cũ được xóa khi Start	Xử lý các Round, kết quả trung gian chưa hợp lệ
Hoàn tất	Chốt `cipher_latched`; cập nhật DONE	Ready báo hoàn thành; Ciphertext hợp lệ
Trả kết quả	Đọc CIPHER0–3, ghép lại 128 bit	Giữ kết quả đến tác vụ mới

Giới hạn của cờ DONE trong phiên bản RTL đang mô tả: Trong AXI Controller hiện tại, `done_latched` được xóa khi ghi Start nhưng điều kiện chốt kết quả kiểm tra `aes_ready_i=1`, chưa có xác nhận riêng cạnh Ready sau Busy. Trường hợp Ready còn cao ngay sau một lệnh mới có thể tạo DONE sớm và đưa Ciphertext cũ ra thanh ghi kết quả. Các chương trình kiểm chứng thực tế đã đạt PASS với các vector thử, nhưng điều đó không tương đương chứng minh hoàn toàn không có race giữa xung Start và READY. Một cải tiến hợp lý là thêm cờ `seen_busy`: chỉ chốt DONE khi đã quan sát READY xuống 0 của giao dịch hiện hành và sau đó trở lên 1. Đây là *hướng hiệu chỉnh*, không phải một khối đã được xác nhận tồn tại trong source dùng tạo Bitstream.

Kiểm tra tính nguyên tử của dữ liệu. Vì KEY/PLAIN là nhiều thanh ghi 32 bit, phần mềm không được ghi đè bất kỳ word nào trong khi Core đang xử lý. Tương tự, với Replica, `rm_select` phải được giữ ổn định cho đến khi Ciphertext được chốt. Những quy tắc này kết hợp với WSTRB và thứ tự word little-endian tại Register Map tạo nên hợp đồng giao tiếp giữa phần mềm C và RTL; nếu sai một vị trí word, NIST có thể thất bại dù bản thân Round Datapath thực hiện đúng thuật toán.

Tổ chức tín hiệu trong một giao dịch MMIO. Có hai đường thời gian khác nhau. Trên bus, một thao tác ghi hợp lệ hoàn tất khi các kênh AW, W và B thực hiện xong Handshake. Ở lõi, một lần mã hóa bắt đầu khi xung `aes_start_o` được chấp nhận tại cạnh `clk_sys`. Các mốc này không được đồng nhất nếu không có bằng chứng hai Clock nối chung và giao dịch được chốt cùng một cạnh. Cách ghi STATUS trong chương trình C vì vậy dựa vào trạng thái sau khi phát Start, thay vì suy diễn rằng AXI Write Response chính là AES Done.



Hình 12: Trình tự tương tác PS–AXI–AES

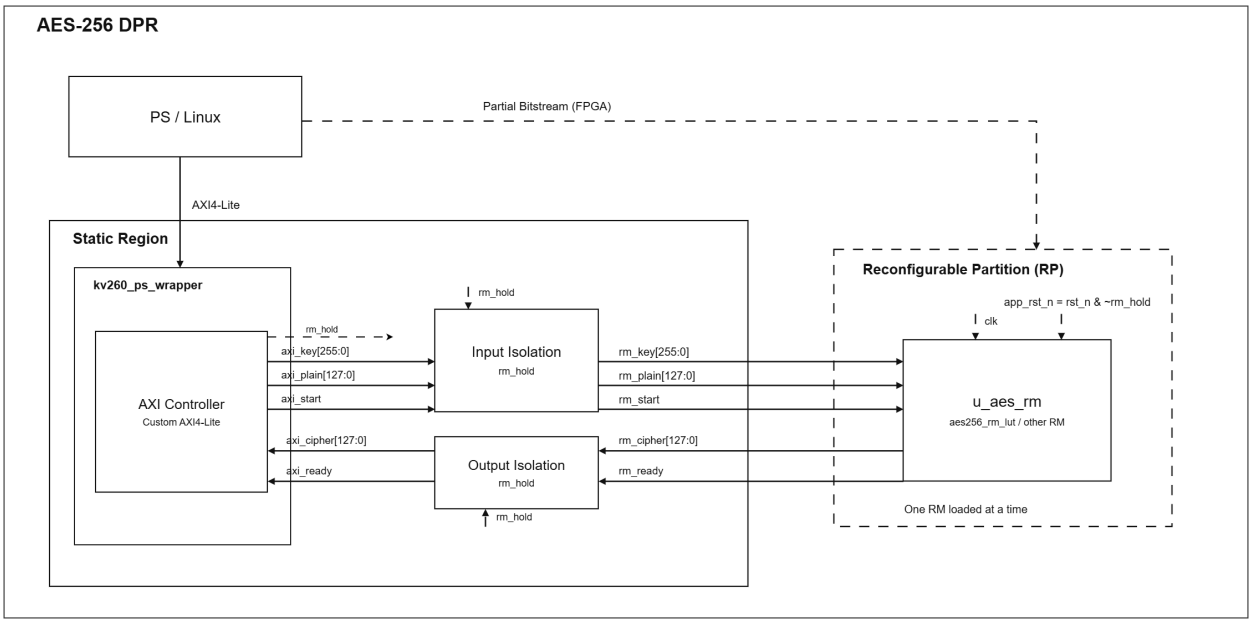
Hai tín hiệu `STATUS[0]` và `STATUS[1]` được đọc trong cùng từ 32 bit nhưng khác bản chất: DONE là cờ chốt trong Controller, còn READY phản ánh Core. Kiến trúc mong muốn chỉ báo DONE cho một giao dịch sau khi nhận đúng pha hoàn tất của giao dịch ấy. Đây là một điểm thiết kế giao tiếp đáng phân tích riêng, bởi bộ NIST chỉ so sánh Ciphertext cuối và không thay thế kiểm chứng mọi thứ tự Timing có thể xảy ra của bus AXI.

3.5 Thiết kế kiến trúc DPR và Replica

Sau khi thống nhất giao diện AES-256 và lớp thanh ghi AXI4-Lite, nhóm xây dựng hai phương án tích hợp các biến thể trên KV260. Phương án Dynamic Partial Reconfiguration (DPR) sử dụng một Reconfigurable Partition (RP) để lần lượt triển khai các Reconfigurable Module (RM). Trong khi đó, phương án Replica giữ bốn lõi AES-256 thường trú trong PL và lựa chọn lõi hoạt động bằng tín hiệu điều khiển. Nội dung mục này tập trung vào tổ chức Top-level, các đường kết nối và cơ chế chuyển đổi được hiện thực trong RTL. Nguyên lý chung của DPR và Replica đã trình bày ở Chương 2, còn việc tạo bitstream và triển khai trên FPGA được trình bày ở Chương 4.

3.5.1 Kiến trúc Top-level DPR

Kiến trúc DPR được mô tả tại `top_dfx_aes256.sv`. Thành phần `kv260_ps_wrapper` cung cấp Clock, Reset, dữ liệu và tín hiệu điều khiển từ hệ thống PS–AXI. Về phía giao diện AES, ba tín hiệu xuất từ phần tĩnh gồm `axi_key[255:0]`, `axi_plain[127:0]` và `axi_start`; chiều ngược lại sử dụng `axi_cipher[127:0]` và `axi_ready`. Các tín hiệu này đi qua lớp cách ly trước khi kết nối với RM. Nhờ đó, các thanh ghi AXI và phần mềm điều khiển không phải thay đổi khi thay thế một biến thể AES-256 bằng biến thể khác.



Hình 13: Kết nối Top-level DPR với đường dữ liệu qua logic cách ly và một Reconfigurable Partition.

Trong Hình 13, RP chứa thể hiện `u_aes_rm`. Đây là tên instance được giữ cố định trong Top-level để phục vụ việc gắn với định nghĩa phân vùng tái cấu hình trong quy trình DFX. Phiên bản RTL đang sử dụng `aes256_rm_lut` tại vị trí này; các wrapper `aes256_rm_bram`, `aes256_rm_composite` và `aes256_rm_dummy_noise` có cùng tập cổng, dùng làm các phương án RM thay thế. Mỗi wrapper nhận `clk_sys`, `rst_n`, Key, Plaintext và Start, đồng thời trả về Ciphertext cùng Ready. Tại một thời điểm vận hành, RP chỉ chứa RM tương ứng với cấu hình đã nạp, không phải bốn lõi hoạt động đồng thời.

Lớp cách ly được điều khiển bằng `rm_hold`, lấy từ thanh ghi `RM_CONTROL[0]`. Khi tín hiệu này bằng 1, RTL đưa `rm_key`, `rm_plain`, `rm_start` về 0 và cũng chặn Ciphertext, Ready từ RM ở chiều phản hồi. Khi `rm_hold` bằng 0, các đường dữ liệu được nối thông tương ứng giữa phía AXI và RM. Bên cạnh đó, Reset cấp vào RM được xác định theo:

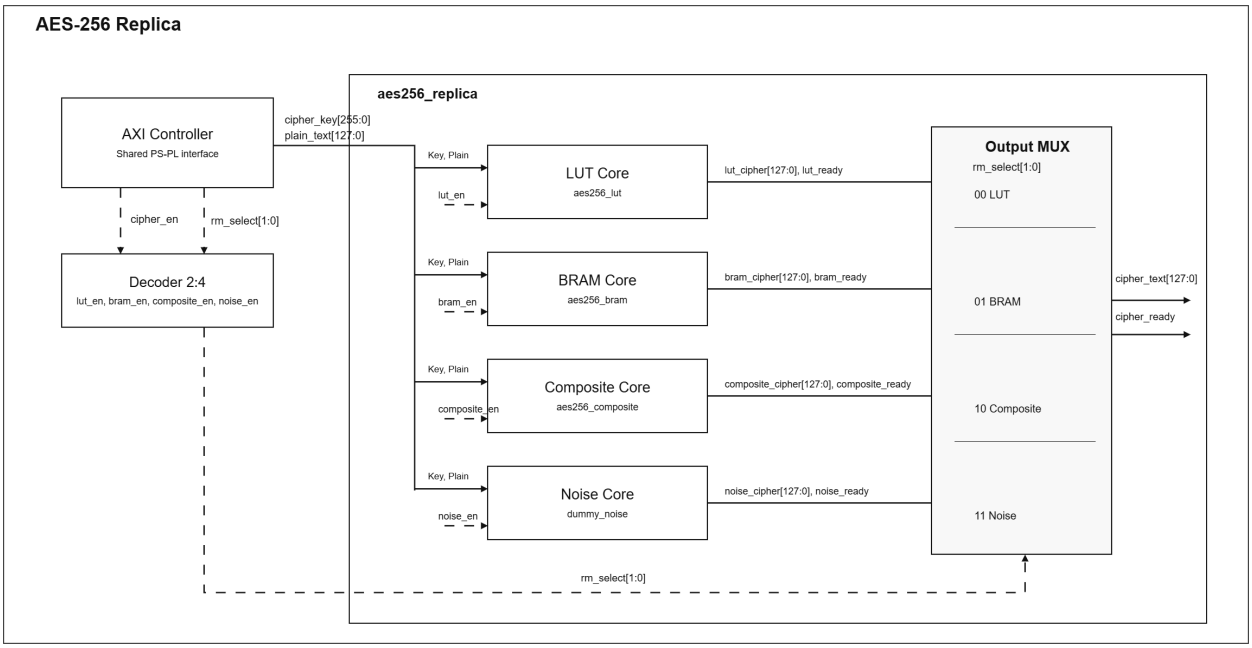
$$\texttt{app_rst_n} = \texttt{rst_n} \ \wedge \ \neg \texttt{rm_hold}.$$

Do đó, đưa `rm_hold` lên 1 vừa đặt RM về trạng thái Reset, vừa ngăn các tín hiệu không xác định từ vùng đang thay đổi lan sang các khối điều khiển tĩnh. Phần AXI vẫn duy trì hoạt động để phần mềm tiếp tục truy cập thanh ghi. Đây là cơ chế bảo vệ giao diện ở mức RTL, không thay thế các yêu cầu về cách ly và kiểm tra tính hợp lệ của thiết kế trong DFX flow.

Việc lựa chọn biến thể của DPR không thực hiện bằng `rm_select`. Phần mềm trên PS quản lý tệp Partial Bitstream và yêu cầu hệ thống nạp cấu hình mới thông qua FPGA Manager. Một trình tự điều khiển thích hợp là kết thúc tác vụ đang xử lý, đặt `rm_hold=1`, nạp cấu hình RM mới, đưa `rm_hold=0` và chỉ sau đó phát Start. Như vậy, đường nạp cấu hình là luồng điều khiển riêng với đường AXI4-Lite truyền Key, Plaintext và Ciphertext. Mục 3.5 chỉ mô tả cấu trúc kết nối; các bước tạo RP, bố trí Pblock và nạp Partial Bitstream thực tế sẽ được trình bày ở Chương 4.

3.5.2 Kiến trúc Top-level Replica

Phương án Replica được mô tả trong module `aes256_replica.sv`, khởi tạo bốn lõi độc lập: `u_lut`, `u_bram`, `u_composite` và `u_noise`. Cả bốn cùng nhận Key 256 bit, Plaintext 128 bit, Clock và Reset. Tuy nhiên, tín hiệu Start không được phân phối đồng thời đến tất cả các lõi. RTL giải mã `rm_select[1:0]` để chỉ cấp xung Start cho biến thể đang được chọn; các lõi còn lại không nhận yêu cầu mã hóa mới.



Hình 14: Kết nối các lõi AES-256 trong kiến trúc Replica với logic giải mã Start và lựa chọn đầu ra.

Ở Hình 14, đường Key và Plaintext được dùng chung, còn bốn tín hiệu `lut_en`, `bram_en`, `composite_en` và `noise_en` là các nhánh Start riêng. Các nhánh nét đứt được nhận diện bằng nhãn net tương ứng để tránh tạo nhiều đường giao nhau trên hình. Điều kiện kích hoạt từng lõi được viết gọn như sau:

$$\texttt{en}_i = \texttt{cipher_en} \ \wedge \ (\texttt{rm_select} = i), \quad i \in \{0, 1, 2, 3\}.$$

Tại đầu ra, mỗi lõi cung cấp cặp tín hiệu Ciphertext và Ready độc lập. Khối Output MUX chọn đồng thời hai tín hiệu của cùng một lõi theo giá trị `rm_select[1:0]`. Ánh xạ lựa chọn trong RTL được thể hiện tại Bảng 15.

Bảng 15: Ánh xạ tín hiệu chọn biến thể của kiến trúc Replica.

`rm_select`	**Lõi nhận Start**	**Cặp tín hiệu đầu ra được chọn**
2'b00	LUT	`lut_cipher`, `lut_ready`
2'b01	BRAM	`bram_cipher`, `bram_ready`
2'b10	Composite Field	`composite_cipher`, `composite_ready`
2'b11	Dummy Noise	`noise_cipher`, `noise_ready`

Bộ chọn đầu ra được mô tả bằng `always_comb`, vì vậy thay đổi `rm_select` làm thay đổi trực tiếp đường Ciphertext và Ready đang quan sát, không có thanh ghi chốt lựa chọn trong `aes256_replica`. Để tránh đọc nhầm kết quả của lõi khác, phần mềm cần xác lập giá trị chọn trước khi Start và giữ nguyên cho đến khi yêu cầu mã hóa hiện tại hoàn tất. Trong kiến trúc này, việc đổi biến thể không đòi hỏi nạp Partial Bitstream mới, nhưng mọi lõi vẫn hiện diện trong thiết kế nên chi phí tài nguyên đồng thời có thể cao hơn DPR. Mức sử dụng LUT, FF, BRAM và thời gian thực thi thực tế sẽ được xác định bằng kết quả tổng hợp và đo kiểm ở các chương sau.

Trong bộ RTL tham chiếu ban đầu, tên `aes256_composite` được gọi ở cấp Replica trong khi một file mô tả khai báo module `aes_composite`. Đây là chi tiết phải đối chiếu với danh sách source thực sự sử dụng khi tạo Bitstream. Việc Replica đã Implementation và chạy NIST thành công cho thấy cấu hình nạp trên KV260 có kết nối chức năng phù hợp; từ các file `.tex` hiện có không thể xác định chính xác nhóm đã đổi tên module hay thêm wrapper trong phiên bản build cuối.

3.5.3 Giao diện chung và cơ chế chuyển đổi biến thể

Hai kiến trúc sử dụng cùng định dạng Key 256 bit, Plaintext 128 bit, Ciphertext 128 bit và cặp tín hiệu Start–Ready. Vì vậy, chương trình trên PS có thể tiếp tục sử dụng Register Map ở Mục 3.4 cho cả hai trường hợp. Sự khác biệt nằm ở thao tác chuẩn bị phần cứng trước mỗi yêu cầu mã hóa: DPR phải bảo đảm đã nạp đúng RM và giải phóng cách ly, còn Replica chỉ cần đặt lựa chọn MUX phù hợp. Bảng 16 tổng hợp các điểm khác nhau trực tiếp thể hiện trong Top-level RTL.

Bảng 16: So sánh tổ chức và cơ chế chuyển đổi tại mức Top-level RTL.

Nội dung	**DPR**	**Replica**
Lõi AES trong PL	Một `u_aes_rm` tại RP ở một thời điểm	Bốn thể hiện AES tồn tại trong RTL đồng thời
Lựa chọn biến thể	Nạp Partial Bitstream tương ứng từ PS	`rm_select[1:0]` điều khiển Start Decoder và Output MUX
`RM_HOLD`	Dùng để cách ly đầu vào, đầu ra và giữ Reset RM	Không tham gia chọn lõi trong `aes256_replica`
Thao tác trước Start	Hoàn tất nạp RM, bỏ Hold, sau đó mới Start	Ghi Select, giữ ổn định khi xử lý, sau đó Start
Điều kiện đổi biến thể	Cần thời gian nạp lại cấu hình RM	Đổi đường chọn logic, không cần tái cấu hình PL
Giao diện phần mềm	KEY, PLAIN, COMMAND, STATUS, CIPHER	KEY, PLAIN, COMMAND, STATUS, CIPHER

Việc duy trì cùng giao diện phần mềm là cơ sở để sử dụng các vector kiểm chứng và quy trình thu thập kết quả thống nhất khi so sánh hai kiến trúc. Tuy nhiên, thao tác chọn biến thể chưa đủ để khẳng định một yêu cầu mới đã hoàn tất: phần mềm vẫn cần tuân thủ cơ chế xác nhận trạng thái mã hóa và các giới hạn của logic DONE đã phân tích ở Mục 3.4.4.

Chuỗi thời gian tại ranh giới RP. Cơ chế HOLD không chỉ là một cờ báo phần mềm. Theo Top-level, `rm_hold=1` đồng thời buộc `rm_key`, `rm_plain`, `rm_start` về 0, chặn tín hiệu Ready/Ciphertext từ RM và hạ `app_rst_n`. Như vậy AXI Controller vẫn phản hồi các giao dịch MMIO trong lúc RP chưa sẵn sàng, nhưng không được xem các tín hiệu của RM là dữ liệu mã hóa hợp lệ.

Bảng 17: Tình trạng các đường giao tiếp trong chuỗi chuyển đổi DPR.

Giai đoạn	**RM_HOLD**	**app_rst_n**	**Dữ liệu/điều khiển**
AES đang chạy	0	Theo Reset chung	Key/Plain/Start kết nối với RM
Cô lập trước DFX	1	0	Đầu vào bị ép 0; Cipher/Ready bị che
Nạp Partial Bitstream	1	0	Static AXI tiếp tục hoạt động; RM không nhận Start
RELEASE	0	Theo Reset chung	Giao diện RM được thông nối trở lại
Giao dịch kế tiếp	0	1 khi hết Reset	Phần mềm mới phát Start cho RM đã nạp

DPR không dùng `rm_select` để đổi chức năng trong RP: mã lựa chọn được xác định bởi nội dung Partial Bitstream đang hoạt động. Ngược lại, Replica không thay đổi cấu hình PL. Chỉ lõi được chọn nhận xung `cipher_en`, nhưng các Core không được chọn vẫn nhận Clock theo kết nối trong Top-level. Do đó, *không có*

Start mới không có nghĩa các nhánh không được chọn bị Clock-Gating hoàn toàn. Để kết luận về Dynamic Power của từng Core phải có phân tích hoạt động hoặc dữ liệu công suất phù hợp.

Bảo toàn lựa chọn trong suốt giao dịch. Khác với bus Key/Plaintext được dùng chung, Output MUX Replica là logic tổ hợp theo `rm_select`. Nếu Selection bị đổi khi Core đang Busy, phần mềm sẽ quan sát Ready hoặc Ciphertext của một Core khác. Vì vậy, điều khiển trên ARM/Linux phải kết thúc tác vụ hiện tại trước khi chuyển `RM_CONTROL[2:1]`. Đây là yêu cầu kiến trúc chứ không phải quy định riêng của Testbench.

Bảng đối chiếu trạng thái các Core trong Replica. Ở Replica, Clock và Reset được phân phối đến mọi Core theo Top-level, còn `cipher_en` được giải mã theo `rm_select`. Khi chọn BRAM chẳng hạn, chỉ nhánh `bram_en` nhận Start; `lut_en`, `composite_en`, `noise_en` không kích hoạt yêu cầu mới. Tuy nhiên, dữ liệu Key và Plaintext vẫn xuất hiện trên các bus dùng chung, và các lõi còn lại vẫn có trạng thái đã lưu trước đó. Output MUX đồng thời chọn `bram_ready` và `bram_cipher`. Vì vậy, nếu đổi `rm_select` giữa chừng, kết quả và trạng thái sẽ chuyển ngay sang Core mới được chọn, dù Core cũ chưa hoàn thành.

Điều này giải thích vì sao giao diện chung chỉ là *khả năng tương thích về cổng và giao thức tác vụ*, không có nghĩa bốn Core có cùng latency. Ví dụ, khi LUT và BRAM cùng nhận bộ vector NIST nhưng được Start tuần tự, bộ điều khiển phải chờ Ready của đúng Core đang được chọn; giả định một hằng số chu kỳ hoàn tất chung sẽ không phản ánh đúng cấu trúc. Trong chương trình ARM/Linux, lệnh lựa chọn biến thể luôn đứng trước việc ghi COMMAND và được giữ nguyên đến khi đọc được Ciphertext mới.

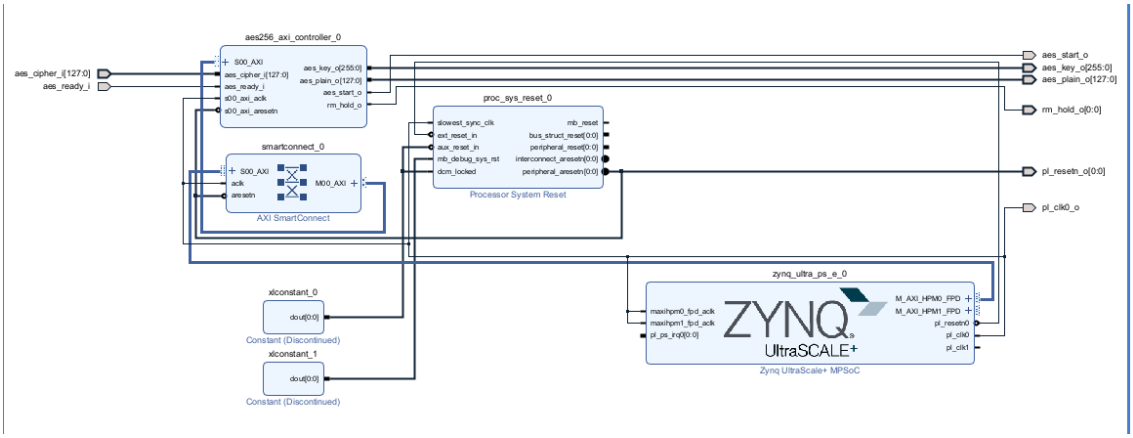
CHƯƠNG 4: HIỆN THỰC TRÊN FPGA VÀ ARM/LINUX

4.1 Môi trường và quy trình hiện thực

Các thiết kế được tổng hợp và triển khai bằng Vivado 2025.2 cho Kria K26 (`xck26-sfvc784-2LV-c`). Quy trình gồm RTL Synthesis, tạo Block Design PS–AXI–PL, Implementation, Routing và xuất file cấu hình. DPR cần thêm việc xác lập RP, Pblock và bốn DFX Configuration; Replica sử dụng một thiết kế tĩnh chứa bốn lõi. Các báo cáo tài nguyên và Timing trong Chương 5 được lấy ở bước sau Routing, không trộn với số liệu trước Implementation [3, 7, 9].

4.2 Tích hợp hệ thống SoC trong Vivado

Block Design sử dụng `zynq_ultra_ps_e_0`, `smartconnect_0`, `proc_sys_reset_0` và Custom AXI4-Lite Controller. Controller chuyển giao dịch 32 bit thành bus Key 256 bit, Plaintext 128 bit và tín hiệu START; dữ liệu Ciphertext trở lại qua các thanh ghi đọc. Hình 15 ghi nhận cấu trúc tích hợp thực tế.

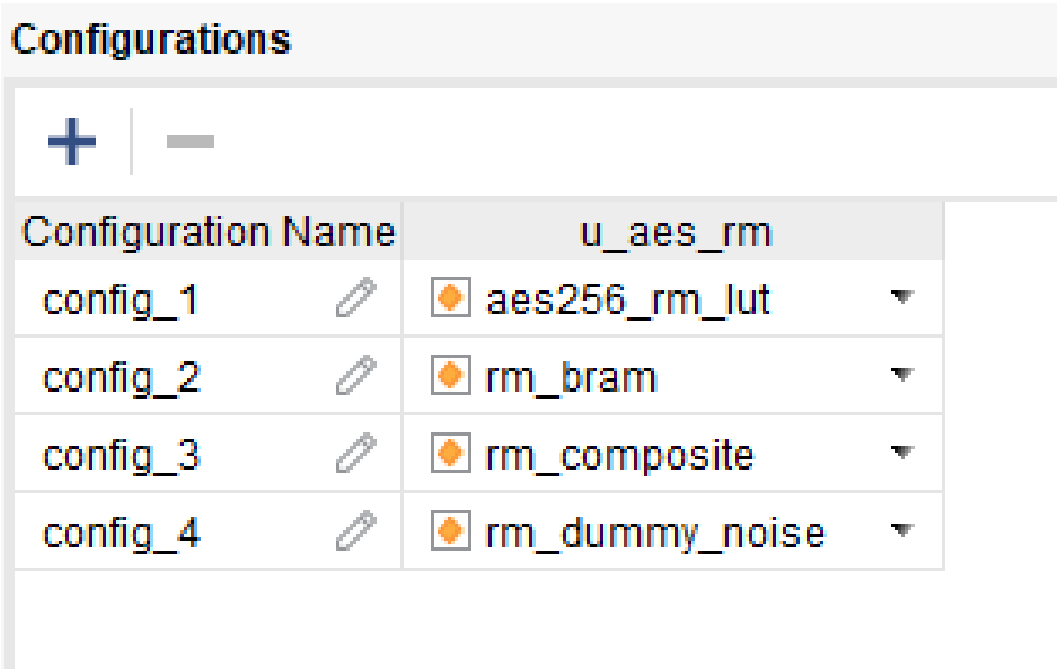


Hình 15: Block Design tích hợp PS, SmartConnect và bộ điều khiển AES-256.

Vùng địa chỉ Slave được gán tại `0xA0000000` với không gian `0x10000` byte. Clock và Reset được cấp từ hệ thống PS–PL; khi đánh giá hai miền Clock, cần dựa vào net kết nối thực tế chứ không suy ra cùng miền chỉ vì cùng tần số ràng buộc. Các địa chỉ và tín hiệu được giữ thống nhất giữa DPR và Replica để dùng cùng chương trình điều khiển.

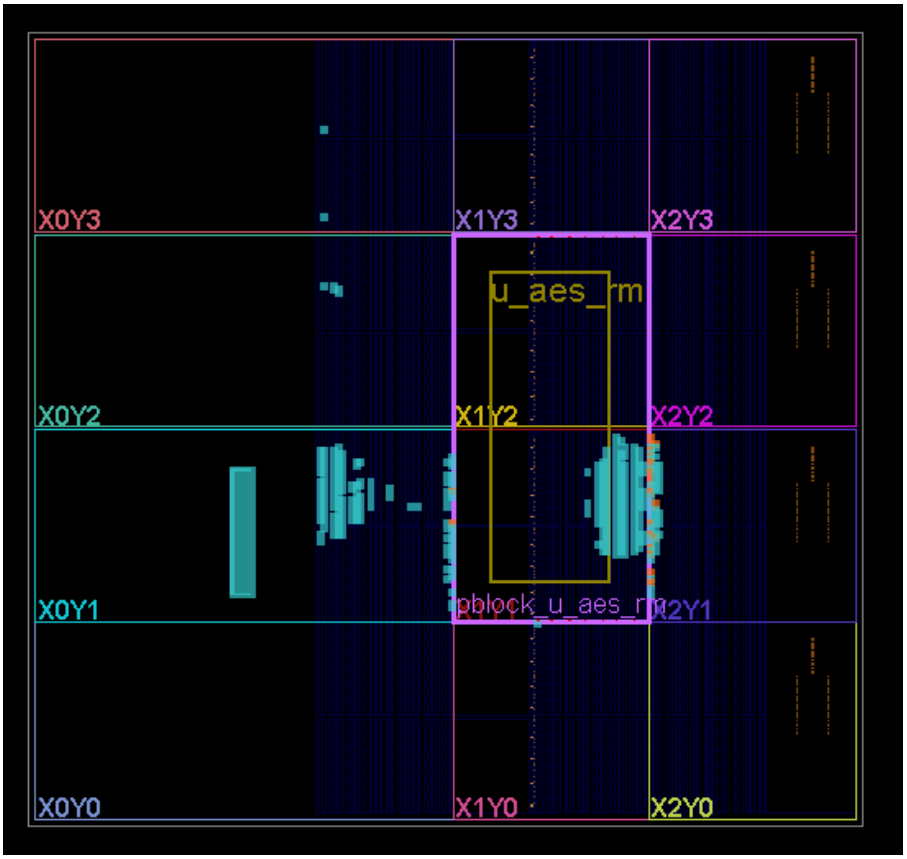
4.3 Hiện thực kiến trúc DPR

Trong DPR, AXI Controller, logic cô lập và các kết nối PS thuộc Static Design; thể hiện `u_aes_rm` là Reconfigurable Partition. Bốn wrapper LUT, BRAM, Composite và Dummy Noise sử dụng chung tập cổng Clock, Reset, Key, Plaintext, Start, Ciphertext và Ready. Trong Vivado DFX, mỗi RM có một Synthesis Run và một Configuration tương ứng. Parent Run của RM khởi tạo được triển khai trước; các cấu hình thay thế được Implementation theo Static Design cố định.



Hình 16: Các DFX Configuration của thiết kế AES-256 DPR trong Vivado.

Pblock được gán cho RP để cung cấp vùng tài nguyên vật lý đáp ứng cả bốn RM, gồm các tài nguyên logic và Block RAM tương ứng. Phần tĩnh được giữ tương thích giữa các Configuration theo cơ chế DFX [9]. Hình 17 minh họa vị trí RP sau Implementation; thiết kế đã hoàn tất Routing và xuất Full Bitstream cùng các Partial Bitstream cần thiết.



Hình 17: Pblock của Reconfigurable Partition trên Kria K26.

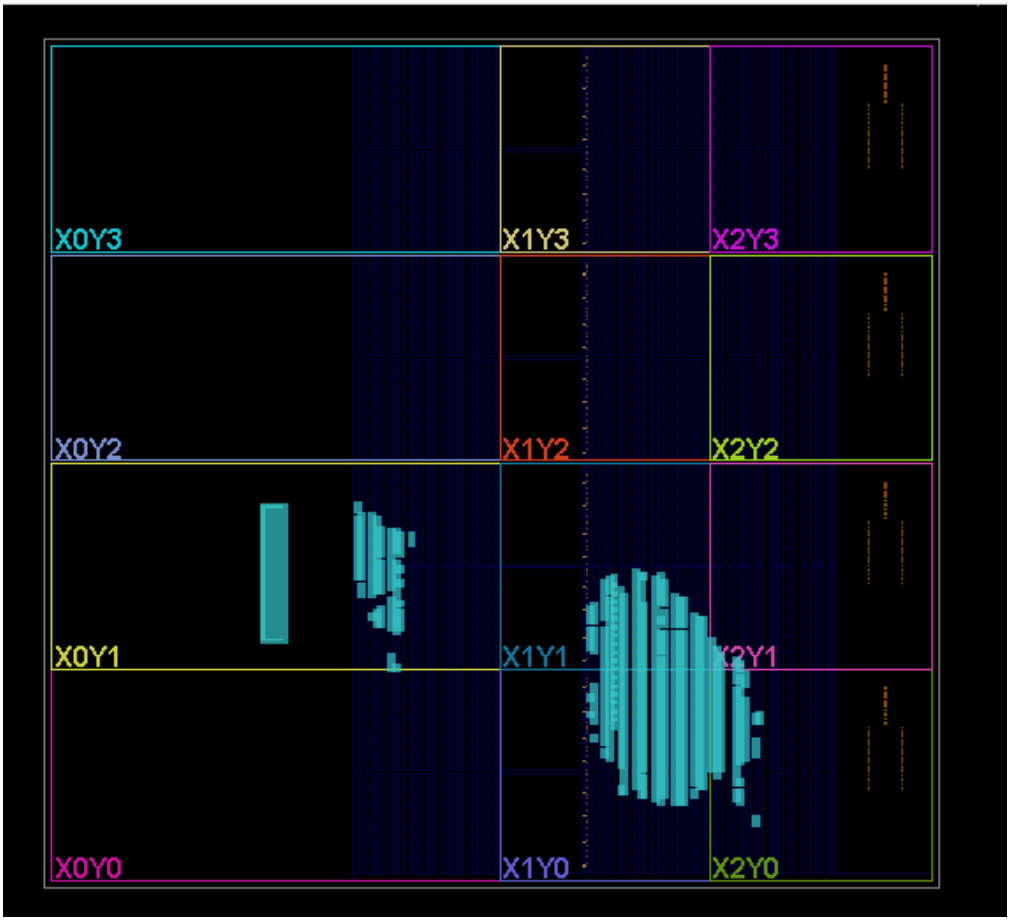
Kiểm tra nhất quán giữa các Configuration. Các hình Design Runs cho thấy các RM có các bước Synthesis và Implementation riêng, nhưng cùng phụ thuộc Static Design và một RP. Khi thiết kế con thay thế không tương thích ranh giới phân vùng, công cụ không thể xem cấu hình đó là một RM hợp lệ chỉ dựa trên việc tên cổng giống nhau ở RTL; quá trình Implementation phải đáp ứng thêm yêu cầu tại ranh giới vật lý. Nhóm xuất các Partial Bitstream sau khi hoàn tất các Configuration trên cùng thiết bị K26.

Name	Configuration	DFX Mode	RM Instance	Constraints	Status
✓ **synth_1** (active)				**constrs_1**	**synth_design Complete!**
✓ **impl_1** (active)	**config_1**	**STANDARD**		**constrs_1**	**route_design Complete!**
✓ child_0_impl_1	config_2	STANDARD			route_design Complete!
✓ child_1_impl_1	config_3	STANDARD			route_design Complete!
✓ child_2_impl_1	config_4	STANDARD			route_design Complete!
Out-of-Context Module Runs					
✓ aes256_rm_lut_synth_1				aes256_rm_lut	synth_design Complete!
✓ rm_bram_synth_1				rm_bram	synth_design Complete!
✓ rm_composite_synth_1				rm_composite	synth_design Complete!
✓ kv260_ps					Submodule Runs Complete
✓ rm_dummy_noise_synth_1				rm_dummy_noise	synth_design Complete!

Hình 18: Parent/Child Runs của các cấu hình DPR trong Vivado.

4.4 Hiện thực kiến trúc Replica

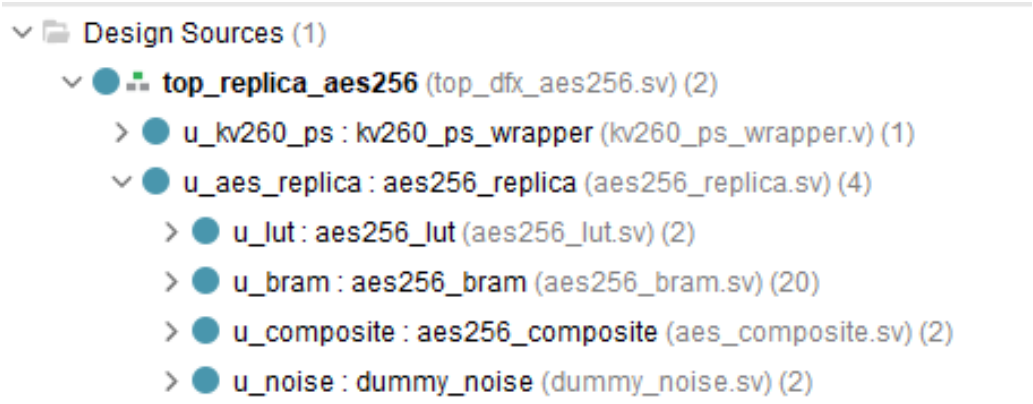
Replica được tổng hợp và Implementation như một cấu hình tĩnh, gồm bốn instance AES-256 và logic Start Decoder/Output MUX. Không có RP hay Child Run; việc chọn lõi do `rm_select[1:0]` quyết định ở thời gian hoạt động. Kết quả Design Runs cho thấy thiết kế hoàn tất Synthesis, Implementation và tạo Bitstream. Device View ở Hình 19 biểu diễn phân bố tài nguyên của toàn hệ thống sau Routing.



Hình 19: Bố trí tài nguyên của thiết kế Replica trên FPGA sau Routing.

Phần tài nguyên của Replica lớn hơn từng cấu hình DPR riêng lẻ do bốn Core cùng hiện diện; số liệu LUT, FF, BRAM và Timing cụ thể được trình bày ở Mục 5.4.

So sánh cách tổ chức hierarchy. Trong DPR, phần hierarchy của một RM tại thời điểm chạy chỉ bao gồm một biến thể. Replica có bốn instance con dưới một Top-level và logic kết nối chọn Core; đây là nguyên nhân trực tiếp dẫn đến khác biệt tổ chức tài nguyên. Hình hierarchy ở mức RTL được dùng để xác nhận các instance tồn tại đồng thời trước khi phân tích tài nguyên sau Routing.



Hình 20: Hierarchy RTL của thiết kế Replica gồm bốn nhánh lõi AES.

4.5 Triển khai trên Kria KV260

Các Bitstream được triển khai trên KV260 chạy Ubuntu 22.04.5 LTS (Kernel Xilinx ZynqMP). Ứng dụng FPGA được tổ chức trong `/lib/firmware/xilinx/` với Full Bitstream, Device Tree Overlay và `shell.json`. DPR sử dụng `aes256_dfx.bit.bin` ban đầu và bốn Partial Bitstream cỡ 1.56 MB; Replica sử dụng `aes256_replica.bit.bin` cỡ 7.80 MB. Hai Full Bitstream cùng kích thước file nhưng không đồng nghĩa cùng tài nguyên RTL.

Với DPR, chương trình đặt `RM_HOLD=1`, nạp Partial Bitstream bằng FPGA Manager, kiểm tra trạng thái `operating`, rồi nhả HOLD. Cả bốn biến thể LUT, BRAM, Composite và Noise đều có log nạp thành công. Đối với Replica, ứng dụng `aes256_replica` được nạp qua `xmutil`; danh sách ứng dụng xác nhận `Active_slot=0` của Replica và `-1` của DPR tại thời điểm kiểm tra.

```
sudo /usr/sbin/fpgautil \
-b /home/ubuntu/aes256_bram_partial.bit.bin \
-f Partial 2>&1 | tee bram_fpgautil.log

sudo dmesg | grep -Ei "fpga|firmware|bitstream|zynq" \
| tee bram_kernel.log
Time taken to load BIN is 8.000000 Milli Seconds
BIN FILE loaded through FPGA manager successfully
[ 1468.776519] fpga_manager fpga0: writing aes256_bram_partial.bit.bin to Xilinx ZynqMP FPGA Manager
ubuntu@kria:~/dpr/logs$ sudo dmesg -C

sudo /usr/sbin/fpgautil \
-b /home/ubuntu/aes256_composite_partial.bit.bin \
-f Partial 2>&1 | tee composite_fpgautil.log

sudo dmesg | grep -Ei "fpga|firmware|bitstream|zynq" \
| tee composite_kernel.log
Time taken to load BIN is 8.000000 Milli Seconds
BIN FILE loaded through FPGA manager successfully
[ 1477.052211] fpga_manager fpga0: writing aes256_composite_partial.bit.bin to Xilinx ZynqMP FPGA Manager
ubuntu@kria:~/dpr/logs$ sudo dmesg -C

sudo /usr/sbin/fpgautil \
-b /home/ubuntu/aes256_lut_partial.bit.bin \
-f Partial 2>&1 | tee lut_fpgautil.log

sudo dmesg | grep -Ei "fpga|firmware|bitstream|zynq" \
| tee lut_kernel.log
Time taken to load BIN is 8.000000 Milli Seconds
BIN FILE loaded through FPGA manager successfully
[ 1480.666052] fpga_manager fpga0: writing aes256_lut_partial.bit.bin to Xilinx ZynqMP FPGA Manager
ubuntu@kria:~/dpr/logs$
obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
```

Hình 21: Log nạp Partial Bitstream DPR trên KV260.

```
  ZERO_PLAINTEXT: CT=F29000B62A499FD0A9F39A6ADD2E7780  PASS
ubuntu@kria:~/replica$ sudo xmutil listapps
cat /sys/class/fpga_manager/fpga0/state
                Accelerator         Accel_type                      Base          Base_type     #slots(PL+AIE)        Active_slot
              aes256_replica          XRT_FLAT              aes256_replica          XRT_FLAT            (0+0)                  0,
                  aes256_dfx          XRT_FLAT                  aes256_dfx          XRT_FLAT            (0+0)                  -1
            k26-starter-kits          XRT_FLAT            k26-starter-kits          XRT_FLAT            (0+0)                  -1
operating
ubuntu@kria:~/replica$
obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
```

Hình 22: Trạng thái ứng dụng Replica đang hoạt động theo `xmutil`.

Trạng thái FPGA Manager `operating` chỉ xác nhận quá trình quản lý cấu hình ở mức driver. Việc xác định đúng ứng dụng đang chạy còn dựa vào `xmutil listapps` và kết quả mã hóa ở Chương 5.

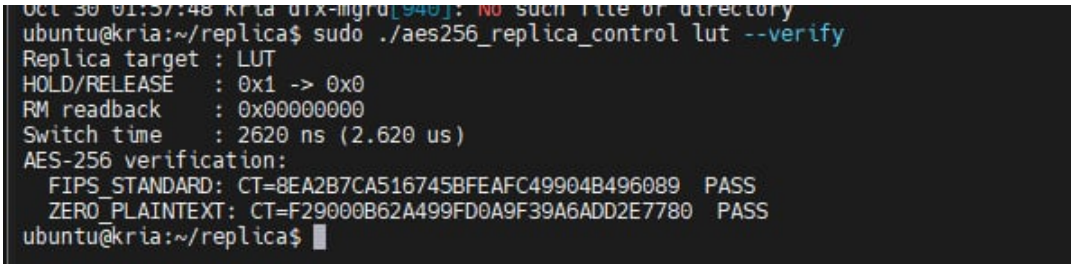
4.6 Chương trình điều khiển ARM/Linux

Nhóm xây dựng hai chương trình C `aes256_dpr_control.c` và `aes256_replica_control.c`. Cả hai mở `/dev/mem`, ánh xạ vùng AXI bằng `mmap()` và đọc/ghi thanh ghi 32 bit qua con trỏ `volatile`. Key 256 bit được ghi thành tám word, Plaintext thành bốn word; phần mềm ghi `COMMAND[0]=1`, chờ trạng thái hoàn tất rồi đọc bốn word Ciphertext.

Bảng 18: Nhóm thanh ghi được sử dụng bởi ARM/Linux.

Offset	Thanh ghi	Chức năng
0x00, 0x04	COMMAND, STATUS	START; DONE và READY
0x08	RM_CONTROL	HOLD (bit 0), SELECT (bit [2:1])
0x10-0x2C	KEY0–KEY7	Key 256 bit
0x30-0x3C	PLAIN0–3	Plaintext 128 bit
0x40-0x4C	CIPHER0–3	Ciphertext 128 bit

DPR Control thực hiện cô lập, gọi `fpgautil` với Partial Bitstream, kiểm tra mã trả về và giải phóng HOLD trước khi chạy AES. Replica Control chỉ chọn lõi bằng cặp giá trị HOLD/RELEASE: LUT `0x1/0x0`, BRAM `0x3/0x2`, Composite `0x5/0x4` và Noise `0x7/0x6`. Cả hai chương trình có tùy chọn `-verify` dùng hai vector đại diện để kiểm tra đường điều khiển và Ciphertext.



Hình 23: Kiểm tra Replica LUT qua ARM/Linux với hai vector AES-256 PASS.

Các số liệu thời gian hiển thị khi gọi chương trình Control chỉ minh họa quá trình chạy; phép đo switching latency và workload có định nghĩa riêng trong Chương 6.

CHƯƠNG 5: KIỂM CHỨNG VÀ ĐÁNH GIÁ AES-256

5.1 Phương pháp kiểm chứng

Việc kiểm chứng được thực hiện ở ba mức: RTL Simulation để kiểm tra trình tự điều khiển, NIST Known Answer Tests trên KV260 để kiểm tra tích hợp PS–AXI–PL, và báo cáo Vivado sau Routing để đối chiếu tài nguyên và Timing. Mỗi testcase được xem là PASS nếu giao dịch hoàn tất trong Timeout và Ciphertext 128 bit khớp giá trị tham chiếu.

Nhóm sử dụng phần `[ENCRYPT]` của bốn bộ vector AESAVS: `ECBGFSbox256` (5 trường hợp), `ECBKeySbox256` (16), `ECBVarKey256` (256) và `ECBVarTxt256` (128), tổng cộng 405 vector cho mỗi Core. Các bộ dữ liệu chỉ kiểm tra Encryption; kết quả không đồng nghĩa chứng nhận CAVP chính thức.

Bảng 19: Phạm vi kiểm chứng NIST AES-256 Encryption.

Test Suite	Vector/Core	DPR (4 RM)	Replica (4 Core)
ECBGFSbox256	5	20	20
ECBKeySbox256	16	64	64
ECBVarKey256	256	1 024	1 024
ECBVarTxt256	128	512	512
Tổng	**405**	**1 620**	**1 620**

5.2 Kiểm chứng bằng RTL Simulation

Các Testbench của LUT, BRAM, Composite Field và Dummy Noise sử dụng cùng một AES-256 Known Answer Test với Clock 10 ns: Key `000102...1F`, Plaintext `001122...FF`, Ciphertext tham chiếu được sử dụng là:

`8EA2B7CA516745BFEAFC49904B496089`

Cả bốn biến thể trả về Ciphertext đúng. Theo cùng cách đếm từ xung `cipher_en` đến `cipher_ready` trong Testbench, LUT, Composite và Noise ghi nhận 15 chu kỳ, còn BRAM ghi nhận 29 chu kỳ. Kết quả phù hợp với việc BRAM cần thêm pha đọc S-box đồng bộ; nhánh CPNG của Noise hoạt động song song với đường mã hóa nên không làm tăng số chu kỳ trong thử nghiệm này. Các con số trên là độ trễ theo Testbench, không phải thời gian truy cập AXI hoặc độ trễ phần mềm đo trên KV260.

```
# run 1000ns
==================================================
 AES-256 LUT RTL VERIFICATION
==================================================
KEY       = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PLAINTEXT= 00112233445566778899aabbccddeeff
EXPECTED = 8ea2b7ca516745bfeafc49904b496089
--------------------------------------------------
ACTUAL    = 8ea2b7ca516745bfeafc49904b496089
LATENCY   = 15 clock cycles
[PASS] AES-256 LUT ciphertext correct
OVERALL RESULT : PASS
==================================================
```

```
# run 1000ns
==================================================
 AES-256 BRAM RTL VERIFICATION
==================================================
KEY       = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PLAINTEXT = 00112233445566778899aabbccddeeff
EXPECTED  = 8ea2b7ca516745bfeafc49904b496089
--------------------------------------------------
ACTUAL    = 8ea2b7ca516745bfeafc49904b496089
LATENCY   = 29 clock cycles
[PASS] AES-256 BRAM ciphertext correct
OVERALL RESULT : PASS
==================================================
```

Hình 24: Log Testbench LUT và BRAM với cùng vector AES-256; cả hai PASS.

Testbench Replica lần lượt đặt `rm_select=00,01,10,11`, giữ ổn định lựa chọn trong mỗi giao dịch rồi so sánh Ciphertext. Bốn lần chọn Core đều PASS, `error_count=0`. Kết quả này kiểm tra đường chọn Start/Output MUX ở mức RTL, không thay thế kiểm chứng 405 vector trên phần cứng.

Bảng 20: Tóm tắt mô phỏng chức năng.

Testbench	Độ trễ ghi nhận	Kết quả
AES LUT	15 chu kỳ	PASS
AES BRAM	29 chu kỳ	PASS
AES Composite Field	15 chu kỳ	PASS
AES Dummy Noise (CPNG)	15 chu kỳ	PASS
Replica, bốn lựa chọn	Không đo chu kỳ theo bài test này	4/4 PASS

Kết quả 15/29/15/15 chu kỳ được lấy từ các Testbench riêng cho từng Core với một vector tham chiếu; bài kiểm tra lựa chọn Replica là phép thử khác, chỉ xác nhận tính đúng đắn của đường chọn Core.

5.3 Kiểm chứng chức năng trên Kria KV260

Hai chương trình `aes256_dpr_nist.c` và `aes256_replica_nist.c` đọc bốn file NIST `.rsp`, ghi Key/Plaintext qua AXI4-Lite, phát Start, chờ trạng thái hoàn tất và so sánh 128 bit Ciphertext. Chương trình xử lý Timeout READY/DONE và lưu kết quả theo từng testcase vào CSV. DPR nạp lần lượt bốn Partial Bitstream; Replica đổi `RM_CONTROL` để chọn Core trong Full Bitstream đã xác nhận hoạt động.

Bảng 21: Kết quả NIST trên phần cứng KV260.

Kiến trúc	LUT	BRAM	Composite	Noise	Tổng
DPR	405/405	405/405	405/405	405/405	1 620/1 620
Replica	405/405	405/405	405/405	405/405	1 620/1 620

Kết quả log cho thấy cả hai kiến trúc có **3 240/3 240 PASS**, không có FAIL, READY Timeout hoặc DONE Timeout. Mỗi kiến trúc có 1 620 bản ghi CSV, gồm đủ 405 tổ hợp Test Suite–COUNT trên từng biến thể. Ciphertext được kiểm tra lại với dữ liệu đầu vào tương ứng và không phát hiện sai khác giữa DPR với Replica. Hai hình dưới đây là phần tổng kết thực thi trên bo mạch.

```
============================================================
RM: NOISE
Partial: /home/ubuntu/aes256_dummy_noise_partial.bit.bin
============================================================
DPR load time: 144.302613 ms
  ECBGFSbox256.rsp   : 5 cases processed
  ECBKeySbox256.rsp  : 16 cases processed
  ECBVarKey256.rsp   : 256 cases processed
  ECBVarTxt256.rsp   : 128 cases processed

NOISE summary:
  TOTAL          : 405
  PASS           : 405
  FAIL           : 0
  READY TIMEOUT  : 0
  DONE TIMEOUT   : 0

============================================================
 FINAL SUMMARY
============================================================
LUT        TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
BRAM       TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
COMPOSITE  TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
NOISE      TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
------------------------------------------------------------
GRAND TOTAL      : 1620
GRAND PASS       : 1620
GRAND FAIL       : 0
READY TIMEOUT    : 0
DONE TIMEOUT     : 0
CSV              : aes256_dpr_nist_results.csv
RESULT           : ALL TESTS PASS
============================================================
ubuntu@kria:~/dpr$
baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
```

Hình 25: Tổng kết 1 620 testcase NIST trên DPR.

```
============================================================
CORE: COMPOSITE
HOLD/RELEASE: 0x5 -> 0x4
============================================================
Replica switch time: 0.790 us
RM_CONTROL readback: 0x00000004
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

COMPOSITE summary:
  TOTAL           : 405
  PASS            : 405
  FAIL            : 0
  READY TIMEOUT   : 0
  DONE TIMEOUT    : 0

============================================================
CORE: NOISE
HOLD/RELEASE: 0x7 -> 0x6
============================================================
Replica switch time: 0.780 us
RM_CONTROL readback: 0x00000006
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

NOISE summary:
  TOTAL           : 405
  PASS            : 405
  FAIL            : 0
  READY TIMEOUT   : 0
  DONE TIMEOUT    : 0

============================================================
 FINAL SUMMARY
============================================================
LUT         TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
BRAM        TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
COMPOSITE   TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
NOISE       TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
------------------------------------------------------------
GRAND TOTAL       : 1620
GRAND PASS        : 1620
GRAND FAIL        : 0
READY TIMEOUT     : 0
```

Hình 26: Kết quả lựa chọn Core và tổng kết 1 620 testcase NIST trên Replica.

Các phép thử xác nhận chức năng Encryption sau khi nạp RM hoặc chọn Core. Chương trình NIST kiểm chứng mỗi biến thể sau một lượt chuyển; các giá trị thời gian xuất trong log không phải các mẫu switching latency độc lập. Phân bố chuyển đổi và hiệu năng workload được khảo sát riêng tại Chương 6.

5.4 Đánh giá tài nguyên và Timing sau Implementation

Các số liệu được xuất từ bốn Configuration DPR và một thiết kế Replica sau Routing trên cùng thiết bị K26 với Vivado 2025.2. Bảng tài nguyên phân biệt toàn thiết kế với riêng hierarchy AES; bảng Timing sử dụng cùng ràng buộc Clock 100 MHz.

5.4.1 Mức sử dụng tài nguyên của toàn thiết kế

Bảng 22 trình bày số LUT, Flip-Flop (FF), RAMB18 và DSP ở mức Top-level. Với DPR, mỗi dòng đại diện cho một Configuration hoàn chỉnh gồm vùng Static và một RM cụ thể, không phải bốn lõi được cộng tài nguyên với nhau. Với Replica, bốn lõi cùng tồn tại trong một bitstream và số liệu phản ánh toàn bộ logic đi kèm bộ chọn, điều khiển và khối PS–PL.

Bảng 22: Tài nguyên toàn thiết kế sau Routing trên thiết bị Kria K26.

Phương án / biến thể	LUT	FF	RAMB18	DSP
DPR – LUT	2 596	1 796	0	0
DPR – BRAM	1 890	1 796	10	0
DPR – Composite	2 820	1 788	0	0
DPR – Noise	2 979	2 044	8	0
Replica	6 715	3 346	18	0

Ở cùng cấu hình DPR, BRAM dùng ít LUT nhất (1 890 LUT), nhưng sử dụng 10 RAMB18. Điều này cho thấy thiết kế đã được ánh xạ thực tế sang Block RAM, không chỉ được đặt tên là "BRAM" trong RTL. Ngược lại, LUT và Composite không dùng RAMB18 trong các báo cáo này. Biến thể Noise dùng 8 RAMB18 và có số FF cao hơn ba phương án còn lại, phù hợp với việc chứa thêm phần logic tạo nhiễu công suất có tương quan (CPNG). Tuy nhiên, số tài nguyên không phải phép đo trực tiếp công suất tiêu thụ; để kết luận về công suất cần báo cáo Power hoặc đo trên board.

Replica có 6 715 LUT, 3 346 FF và 18 RAMB18 ở mức hệ thống. Các số liệu lớn hơn bất kỳ cấu hình DPR riêng lẻ nào là hệ quả phù hợp với việc giữ đồng thời bốn biến thể, song không được phép quy toàn bộ chênh lệch cho lõi AES vì phần kết nối và logic phụ trợ của hai Top-level khác nhau.

5.4.2 Phân tách tài nguyên lõi mã hóa và phần giao tiếp

Báo cáo `Utilization by Hierarchy` cho phép chọn đúng instance thuộc lõi. Trong DPR, đó là `u_aes_rm` mang thuộc tính `Reconfigurable`; trong Replica là `u_aes_replica` chứa bốn instance con. Khi chỉ xét riêng phần này, kết quả được tổng hợp ở Bảng 23.

Bảng 23: Tài nguyên theo hierarchy của phần AES, không bao gồm toàn bộ PS–PL.

Hierarchy	LUT	FF	RAMB18	Kiến trúc
`u_aes_rm` – LUT	1 340	397	0	DPR
`u_aes_rm` – BRAM	634	397	10	DPR
`u_aes_rm` – Composite	1 564	389	0	DPR
`u_aes_rm` – Noise	1 723	645	8	DPR
`u_aes_replica` (4 lõi)	3 892	1 827	18	Replica

Điểm đáng chú ý là ở cả bốn Configuration DPR, hierarchy `u_kv260_ps` giữ nguyên mức 1 256 LUT và 1 399 FF. Đây là bằng chứng về phần Static chung được duy trì trong các báo cáo DPR. Mỗi RM đóng góp phần tài

nguyên riêng, chẳng hạn BRAM là 634 LUT và 10 RAMB18, còn Noise là 1 723 LUT, 645 FF và 8 RAMB18. Các số liệu này có ý nghĩa trực tiếp hơn khi thảo luận tác động của cách hiện thực AES so với số lượng toàn thiết kế.

Replica dùng 3 892 LUT, 1 827 FF và 18 RAMB18 tại hierarchy `u_aes_replica`. Các instance con có mức tối ưu khác khi đặt trong DPR và Replica, vì vậy không đồng nhất số LUT của từng Core giữa hai flow.

5.4.3 Kết quả Timing sau Routing

Các báo cáo `report_timing_summary` đều xác nhận hai Clock `clk_pl_0` và `clk_pl_1` có chu kỳ ràng buộc 10.000 ns, tương ứng 100 MHz. Tất cả năm Run ghi `All user specified timing constraints are met`; đồng thời WNS dương, TNS bằng không và không có Setup Failing Endpoints trong phần tổng hợp. Điều này cho thấy các cấu hình đã đáp ứng các ràng buộc Setup được khai báo tại tần số mục tiêu, không có nghĩa 100 MHz là Fmax được xác định bằng một phép quét tần số độc lập.

Bảng 24: Kết quả Timing Summary sau Routing với Clock mục tiêu 100 MHz.

Thiết kế	WNS (ns)	TNS (ns)	WHS (ns)	THS (ns)
DPR – LUT	+4.317	0.000	+0.027	0.000
DPR – BRAM	+3.469	0.000	+0.027	0.000
DPR – Composite	+3.167	0.000	+0.027	0.000
DPR – Noise	+2.981	0.000	+0.027	0.000
Replica	+3.766	0.000	+0.015	0.000

WNS nhỏ nhất trong bốn phương án DPR thuộc Noise (+2.981 ns), còn LUT đạt WNS lớn nhất (+4.317 ns). Replica có WNS +3.766 ns, nằm giữa các giá trị ghi nhận ở DPR. Các con số này mô tả biên độ Timing của *toàn thiết kế dưới các ràng buộc được khai báo*, do đó không đủ để kết luận một AES Core có độ trễ mã hóa theo chu kỳ thấp hơn hay có Throughput thực tế cao hơn. Kết quả WNS cũng chịu tác động của đường Timing tới hạn trong mạng logic và giao tiếp, không nhất thiết chỉ nằm trong Datapath AES.

Mặt khác, WHS dương và THS bằng không ở cả năm Run cho thấy không có vi phạm Hold được thống kê trong phần Timing Summary sau Routing. Đây là một điểm quan trọng của Replica: báo cáo Synthesis trước đó không phản ánh trạng thái cuối cùng của thiết kế; việc đánh giá phải dùng báo cáo đã được định tuyến, nơi WHS đạt +0.015 ns. Các cảnh báo phương pháp luận được Vivado liệt kê ở bước phân tích cần được giữ lại như giới hạn của quá trình đánh giá nếu chúng liên quan đến ràng buộc hoặc cách sử dụng phần tử, thay vì coi kết quả Timing dương là bảo đảm tuyệt đối cho mọi chế độ hoạt động.

Tổng hợp Mục 5.4 cho thấy hai kiến trúc đều đạt mục tiêu Timing 100 MHz trong các báo cáo đã cung cấp, nhưng khác nhau về mức sử dụng tài nguyên: DPR giữ một RM trên cùng vùng tái cấu hình, Replica dành tài nguyên cho cả bốn lõi. Phép đánh giá này chỉ phản ánh chi phí triển khai phần cứng và ràng buộc Timing; thời gian tái cấu hình DPR, thời gian điều khiển ARM/Linux và khả năng hoạt động ổn định qua nhiều lần chuyển đổi cần được kiểm tra bằng dữ liệu đo trên KV260 ở các mục thực nghiệm tiếp theo.

5.5 Nhận xét tổng hợp

RTL Simulation và kiểm chứng NIST trên KV260 xác nhận chức năng mã hóa trên cả bốn biến thể trong phạm vi tập vector đã thử. Kết quả Routing cho thấy năm thiết kế đáp ứng các ràng buộc Timing 100 MHz: WNS dương, TNS bằng 0. DPR sử dụng từ 1 890 đến 2 979 LUT cho từng cấu hình, còn Replica sử dụng 6 715 LUT ở mức Top-level. BRAM chuyển một phần nhu cầu tài nguyên từ LUT sang Block RAM, đồng thời cần thêm pha đọc đồng bộ.

Khác biệt về tổ chức phần cứng dẫn đến hai dạng chi phí: DPR nạp Partial Bitstream khi đổi biến thể; Replica sử dụng chọn lõi qua thanh ghi nhưng giữ bốn Core trong PL. Tài nguyên Vivado và kết quả NIST chưa đủ để định lượng chi phí chuyển đổi hay throughput ứng dụng. Chương 6 trình bày các phép đo này trên cùng workload, đồng thời phân biệt công suất ước lượng với phép đo thực tế.

CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1 Thiết lập thực nghiệm

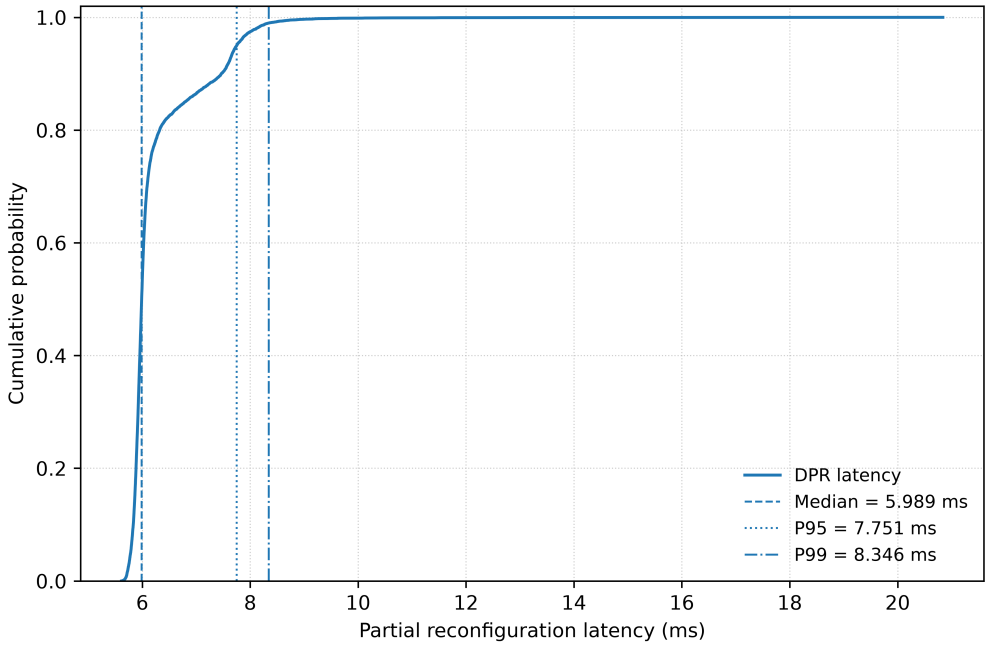
Thực nghiệm chạy trên KV260, Ubuntu 22.04 và thiết kế PL ràng buộc Clock 100 MHz. Hai kiến trúc dùng cùng Register Map AXI4-Lite. Phép thử thứ nhất gồm 10 000 lần chuyển đổi với seed `3152026` và mã hóa kiểm tra sau mỗi lần chuyển. Phép thử thứ hai gồm 10 000 giao dịch AES-256, chia 40 batch, 250 giao dịch/batch, 39 lần chuyển biến thể; DPR và Replica sử dụng cùng chuỗi dữ liệu. Công suất được lấy từ Vivado post-implementation power report, không phải đo từ cảm biến điện trên KV260.

6.2 Đánh giá quá trình chuyển đổi giữa các biến thể

Trong 10 000 lần đổi RM của DPR, cả 10 000 lần nạp và phép kiểm tra AES sau nạp đều thành công. Số lần chọn LUT, BRAM, Composite và Noise lần lượt là 2 485, 2 456, 2 521 và 2 538. Chuỗi lựa chọn dùng cùng seed với Replica. Không có FAIL hoặc Timeout được ghi nhận trong phép thử này.

Bảng 25: Thống kê thời gian nạp Partial Bitstream DPR qua FPGA Manager.

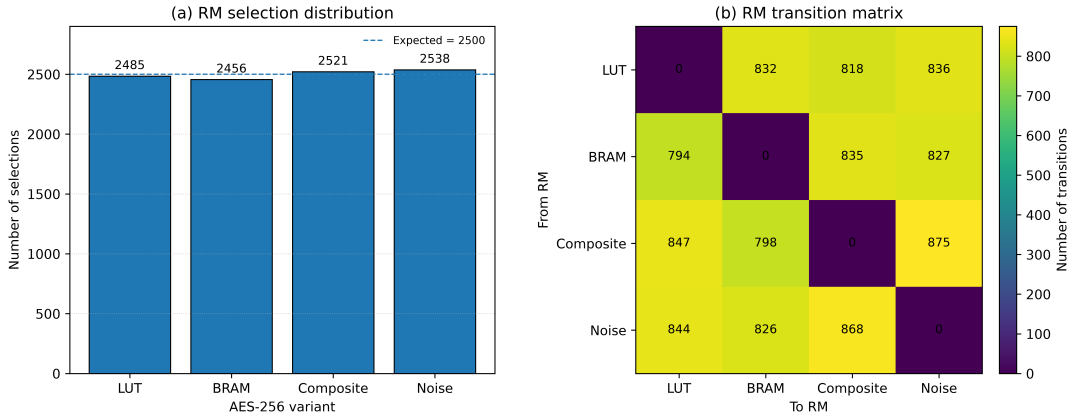
Chỉ tiêu	ms	Chỉ tiêu	ms
Minimum	5.617	Median	5.989
Mean	6.245	Std. deviation	0.688
P95	7.751	P99	8.346
Maximum	20.838	N	10 000



Hình 27: Phân bố tích lũy thời gian nạp Partial Bitstream DPR.

Median khoảng 5.99 ms; 15 mẫu vượt 10 ms nhưng đều hoàn tất. Phép đo khác gọi tiện ích `fpgautil` có trung bình khoảng 45.5 ms do gồm thêm chi phí đường điều khiển Linux. Hai giá trị sử dụng định nghĩa đo khác nhau, không được ghép chung thành một phân bố. Replica lựa chọn Core qua MMIO và ghi nhận trung bình khoảng 0.725 μs. Tỷ số thời gian chuyển đổi ở đây chỉ có ý nghĩa trong các phương pháp đo nêu trên, không phải so sánh độ trễ logic thuần của các Core AES.

Phân bố lựa chọn và hướng chuyển. Ngoài tỷ lệ lựa chọn gần một phần tư cho mỗi RM, log chuyển đổi còn được phân theo 12 hướng từ một biến thể sang ba biến thể khác. Do chuỗi thử nghiệm được thiết kế để đổi sang RM khác, không có chuyển từ một Core về chính nó trong ma trận hướng chuyển. Hình 28 minh họa phân bố này; nó giúp phát hiện liệu phép thử có thiên về một hướng nào đó hay không trước khi diễn giải thống kê latency.



Hình 28: Tần suất lựa chọn RM và các hướng chuyển đổi trong phép thử ngẫu nhiên.

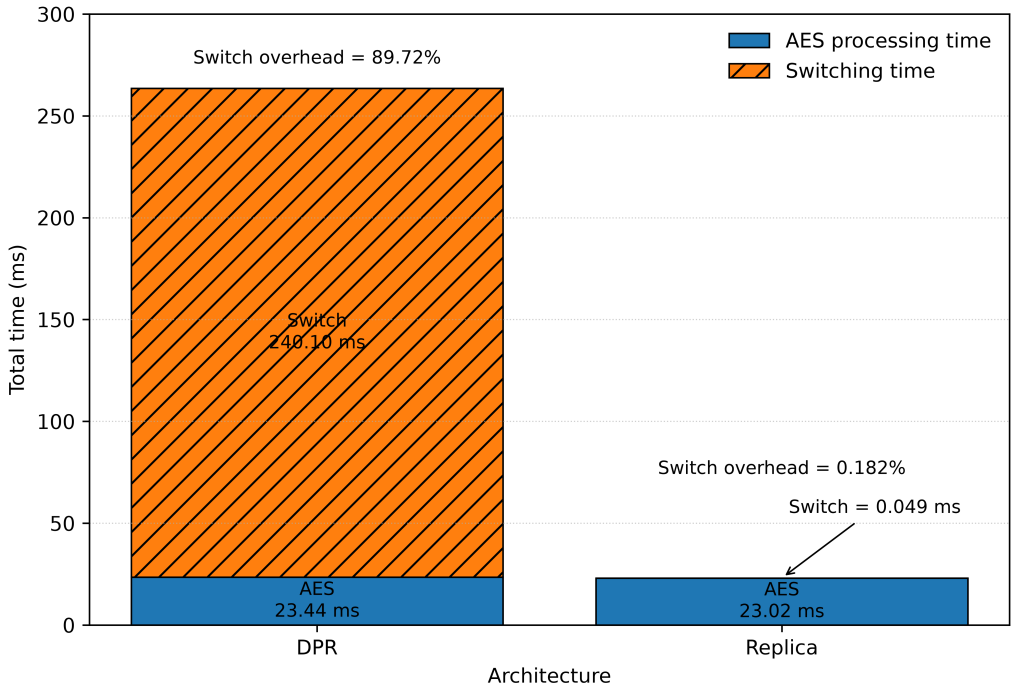
Trong tập dữ liệu ghi nhận, phép kiểm định Chi-square cho phân bố bốn biến thể có $p \approx 0.655$; với 12 hướng chuyển, giá trị khoảng $p \approx 0.744$. Các giá trị này không cho thấy bằng chứng đủ mạnh về sai lệch tần suất trong chuỗi được thử, nhưng cũng không chứng minh PRNG có chất lượng mật mã. Chuỗi chọn biến thể phục vụ tái lập và kiểm tra hệ thống; tính an toàn của lựa chọn ngẫu nhiên không được đánh giá trong đề tài.

6.3 Đánh giá trên cùng workload AES-256

Hai thiết kế xử lý cùng 10 000 giao dịch AES-256, gồm 2 500 giao dịch mỗi biến thể. Workload có 40 batch và 39 lần chuyển đổi. Cả hai kiến trúc đạt 10 000/10 000 PASS. Thời gian xử lý một phép AES trung bình được ghi nhận từ phần mềm, do đó bao gồm ảnh hưởng truy cập thanh ghi/điều phối, không tương đương số chu kỳ thuần của Datapath.

Bảng 26: So sánh workload AES-256 trên cùng chuỗi dữ liệu.

Chỉ tiêu	**DPR**	**Replica**
AES PASS	10 000	10 000
Số chuyển đổi	39	39
AES trung bình (μs)	2.344	2.302
Chuyển đổi trung bình (μs)	6 156.325	1.269
Tổng AES (ms)	23.443279	23.019693
Tổng chuyển đổi (ms)	240.096670	0.049480
Toàn workload (ms)	267.601277	27.122111
Throughput (khối/s)	37 369	368 703
Tỷ trọng thời gian chuyển đổi	89.7218%	0.1824%



Hình 29: Thời gian chuyển đổi và AES trong cùng workload DPR–Replica.

Trong phép thử này, thời gian xử lý AES giữa hai kiến trúc gần nhau; chênh lệch tổng thời gian chủ yếu phát sinh từ chuyển đổi Partial Bitstream. Replica đạt throughput cao hơn khoảng 9.87 lần với batch size 250. Phần thời gian workload không thuộc hai tổng AES/chuyển đổi khoảng 4.06 ms (DPR) và 4.05 ms (Replica), gồm chi phí điều phối phần mềm chưa được đo tách riêng. Nếu tăng batch size, về nguyên tắc chi phí đổi RM được phân bổ trên nhiều giao dịch hơn; nghiên cứu hiện tại chưa có phép đo thực nghiệm trên nhiều batch size để xác định đường quan hệ định lượng.

Độ trễ AES theo biến thể và cách hiểu phép đo. Mỗi biến thể có 2 500 phép mã hóa. Với DPR, LUT, BRAM, Composite và Noise lần lượt đạt thời gian trung bình 2.367, 2.324, 2.330 và 2.357 μs; Replica tương ứng 2.300, 2.301, 2.308 và 2.299 μs. Những giá trị quan sát từ ARM/Linux nằm gần nhau dù Testbench LUT và BRAM có số chu kỳ RTL khác nhau. Nguyên nhân là phép đo phần mềm còn chứa chi phí ghi/đọc các thanh ghi và vòng lặp polling, có thể che khuất chênh lệch nhỏ của phần Core.

Bảng 27: Thời gian AES trung bình theo từng biến thể trong matched workload (μs).

Biến thể	Số giao dịch	DPR	Replica
LUT	2 500	2.367	2.300
BRAM	2 500	2.324	2.301
Composite	2 500	2.330	2.308
Noise	2 500	2.357	2.299
Tổng	**10 000**	**2.344**	**2.302**

Giải thích phần chi phí chuyển đổi. Trong cùng workload, trung bình một lần DPR chuyển RM mất 6 156.325 μs, trong đó riêng giai đoạn lập trình Partial Bitstream khoảng 6 033.713 μs theo bản log phân rã; phần còn lại dành cho HOLD, kiểm tra FPGA Manager, RELEASE và đọc lại thanh ghi. Ở Replica, chọn lõi không truy cập cổng cấu hình FPGA mà chỉ thay đổi giá trị MMIO. Hai cơ chế cùng đạt chức năng mã hóa, nhưng đường điều khiển khác nhau nên chi phí chuyển đổi chênh lệch nhiều bậc độ lớn. Khác biệt throughput

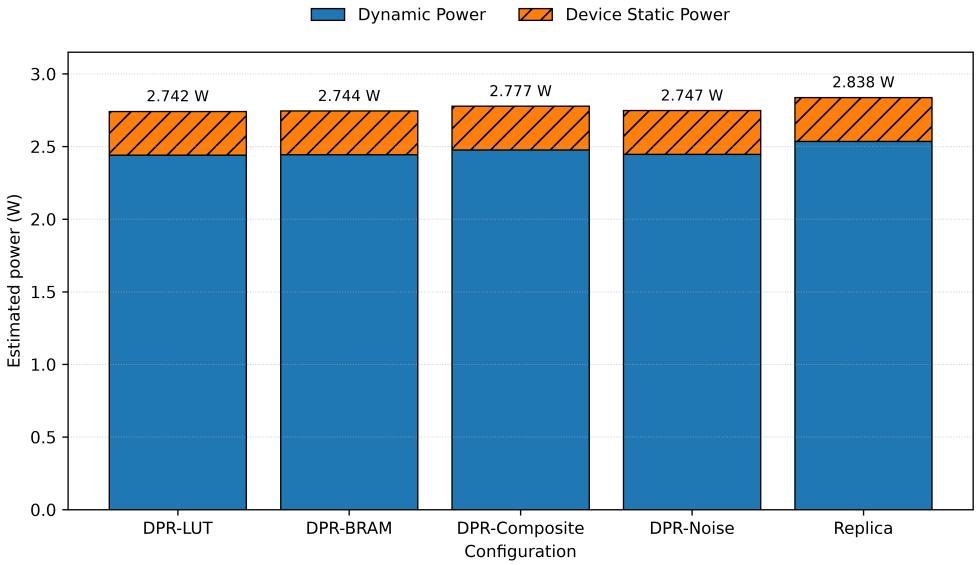
quan sát được không nên quy hoàn toàn cho khả năng tính toán của AES Core.

6.4 Công suất ước lượng sau Implementation

Báo cáo Vivado post-implementation không sử dụng SAIF/VCD và có Confidence Level Medium. Vì vậy, các giá trị sau chỉ dùng để đối chiếu xu hướng của những cấu hình dưới cùng điều kiện ước lượng, không đại diện cho công suất đo trực tiếp trên KV260.

Bảng 28: Ước lượng công suất On-Chip của năm cấu hình.

Cấu hình	Total (W)	Dynamic (W)	Static (W)
DPR–LUT	2.742	2.441	0.301
DPR–BRAM	2.744	2.444	0.301
DPR–Composite	2.777	2.477	0.301
DPR–Noise	2.747	2.447	0.301
Replica	2.838	2.536	0.301



Hình 30: So sánh công suất ước lượng giữa các cấu hình.

Replica có Dynamic Power ước lượng cao hơn bốn cấu hình DPR, phù hợp với việc tích hợp đồng thời bốn Core. Tuy nhiên, do thiếu vết Switching Activity thực tế, các báo cáo không thể phân tách chính xác công suất của từng Core hoặc chứng minh hiệu quả tạo nhiễu đối với phép đo Side-Channel.

6.5 Thảo luận và giới hạn kết quả

Các thực nghiệm xác nhận DPR chuyển đổi giữa bốn RM và duy trì kết quả mã hóa đúng qua 10 000 lượt. DPR dành tài nguyên cho một RM tại mỗi thời điểm nhưng chịu chi phí nạp cấu hình. Replica giữ bốn Core và đổi lựa chọn bằng thanh ghi, vì vậy có throughput cao hơn trong workload 250 khối/batch. Kết quả không chứng minh Replica luôn tối ưu: với batch dài hơn, chi phí DPR có thể được phân bổ trên nhiều khối dữ liệu.

Các số liệu switching latency phụ thuộc đường điều khiển và cách đặt mốc đo. Thời gian phần mềm không phản ánh riêng độ trễ AES Core. Công suất hiện có là ước lượng Vivado, chưa đo trực tiếp trên bo mạch và chưa có phân tích CPA. Kết luận nghiên cứu vì vậy giới hạn ở tính đúng chức năng, khả năng triển khai, tài nguyên và hiệu năng đo được trên KV260 với bộ tham số hiện tại.

TÀI LIỆU THAM KHẢO

[1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197 (FIPS 197), updated May 2023. Available at: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf.

[2] D. Canright, "A Very Compact S-Box for AES," in *Cryptographic Hardware and Embedded Systems – CHES 2005*, Lecture Notes in Computer Science, vol. 3659, pp. 441–455, 2005. doi: https://doi.org/10.1007/11545262_32.

[3] AMD, *Vivado Design Suite User Guide: Synthesis*, UG901, Version 2025.2, December 2025. Available at: https://docs.amd.com/r/2025.2-English/ug901-vivado-synthesis.

[4] AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, UG907, Version 2025.2, November 2025. Available at: https://docs.amd.com/r/2025.2-English/ug907-vivado-power-analysis-optimization.

[5] A. Sajadi, N. Zidaric, T. Stefanov, and N. Mentens, "A Systematic Comparison of Side-channel Countermeasures for RISC-V-based SoCs," in *2024 IEEE Nordic Circuits and Systems Conference (NorCAS)*, pp. 1–7, 2024. doi: https://doi.org/10.1109/NorCAS64408.2024.10752477.

[6] AMD, *Zynq UltraScale+ Device Technical Reference Manual*, UG1085, Rev. 2.5, March 2025. Available at: https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm.

[7] AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, UG906, Version 2025.2, December 2025. Available at: https://docs.amd.com/r/2025.2-English/ug906-vivado-design-analysis.

[8] Arm Limited, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H, 2020. Available at: https://support.arm.com/documentation/ihi0022/h.

[9] AMD, *Vivado Design Suite User Guide: Dynamic Function eXchange*, UG909, Version 2025.2, December 2025. Available at: https://docs.amd.com/r/2025.2-English/ug909-vivado-partial-reconfiguration.

[10] B. Hettwer, J. Petersen, S. Gehrer, H. Neumann, and T. Güneysu, "Securing Cryptographic Circuits by Exploiting Implementation Diversity and Partial Reconfiguration on FPGAs," in *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 260–263, 2019. doi: https://doi.org/10.23919/DATE.2019.8714801.

[11] I. Bow, N. Bete, F. Saqib, W. Che, C. Patel, R. Robucci, C. Chan, and J. Plusquellic, "Side-Channel Power Resistance for Encryption Algorithms Using Implementation Diversity," *Cryptography*, vol. 4, no. 2, Art. no. 13, 2020. doi: https://doi.org/10.3390/cryptography4020013.